

LOYOLA COLLEGE

Chennai – 600 034



Department of Data Science

PRACTICAL RECORD

Course Title : _____

Dept. No. : _____

Name : _____

Semester : _____

Year : _____

Loyola College, Chennai – 600 034



Department of Data Science

BONAFIDE CERTIFICATE

Dept. No. : _____

Name : _____

Class : _____

This is to certify that this is the bonafide record of the practical work done
in _____ at the Loyola College Data Center, during the
year _____.

Head of the Department

Staff-in-Charge

Date : _____

Submitted for the _____ Data Science Practical
examination held on _____ at Loyola College, Chennai – 600 034.

Internal Examiner

External Examiner

CONTENTS

Sl. No.	Title	Date	Page No.
1	Exploratory Data Analysis with Titanic Dataset	01-07-2026	4-10
2	Simple Linear Regression	02-07-2026	11-14
3	Multiple Linear Regression	07-07-2026	15-20
4	Polynomial Regression	10-07-2026	21-25
5	Regularization Techniques	17-07-2026	26-32
6	Logistic Regression	24-07-2026	33-39
7	K-Nearest Neighbour	05-08-2026	40-47
8	Decision Tree Classification	20-08-2026	48-55
9	Random Forest Classification	21-08-2026	56-63
10	Support Vector Machine	25-08-2026	64-69
11	K-Means Clustering	28-08-2026	70-74
12	Hierarchical Clustering	02-09-2026	75-77
13	DBSCAN Clustering	07-09-2026	78-81
14	Principal Component Analysis	15-09-2026	82-85

Exercise – 01

Exploratory Data Analysis with Titanic Dataset

1. AIM

To perform exploratory data analysis on the Titanic dataset by examining its structure, missing values, statistical summaries, distributions, relationships and visual patterns.

2. ALGORITHM

1. Load the Titanic dataset.
2. Inspect rows, dimensions, data types and missing values.
3. Generate statistical summaries.
4. Perform the cleaning and transformations in the notebook.
5. Create the exploratory visualizations.
6. Interpret the observed patterns.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = 'Dataset/titanic.csv'

df = pd.read_csv(data)

df.head()

df.tail()

df.sample(5)

print("Rows and Columns:", df.shape)

df.info()

df.describe()

df.size

df.columns

df.isnull().sum()

missing_percentage = (df.isnull().sum() / len(df)) * 100

print(missing_percentage.sort_values(ascending=False))
```

```

plt.figure(figsize=(10, 6))

sns.heatmap(
    df.isnull(),
    cbar=False,
    yticklabels=False,
    cmap="viridis"
)

plt.title("Missing Values Heatmap")
plt.show()

# Fill Age missing values
df["Age"] = df["Age"].fillna(df["Age"].median())

# Fill Embarked missing values
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Fill Cabin missing values
df["Cabin"] = df["Cabin"].fillna("Unknown")

print(df.isnull().sum())

print("Number of duplicate rows:", df.duplicated().sum())

df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Survived")

plt.title("Passenger Survival Count")
plt.xlabel("Survived")
plt.ylabel("Number of Passengers")

plt.show()

survival_counts = df["Survived"].value_counts()

plt.figure(figsize=(6, 6))

plt.pie(
    survival_counts,
    labels=["Did Not Survive", "Survived"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Survival Distribution")

plt.show()

plt.figure(figsize=(6, 4))

```

```

sns.countplot(data=df, x="Pclass")

plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Sex")

plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Sex",
    hue="Survived"
)

plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Pclass",
    hue="Survived"
)

plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Survived",
    y="Fare"
)

plt.title("Fare Distribution by Survival")
plt.xlabel("Survived")

```

```

plt.ylabel("Fare")

plt.show()

g = sns.catplot(
    data=df,
    x="Pclass",
    hue="Survived",
    col="Sex",
    kind="count",
    height=5,
    aspect=1
)

g.fig.suptitle(
    "Survival by Passenger Class and Gender",
    y=1.05
)

plt.show()

plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Age",
    y="Fare",
    hue="Survived",
    style="Sex",
    size="Pclass",
    sizes=(50, 200)
)

plt.title("Age vs Fare by Survival, Gender and Class")
plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()

numeric_columns = [
    "Survived",
    "Pclass",
    "Age",
    "SibSp",
    "Parch",
    "Fare"
]

correlation = df[numeric_columns].corr()

print(correlation)

plt.figure(figsize=(9, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",

```

```

linewidths=0.5
)

plt.title("Correlation Heatmap")

plt.show()

```

4. OUTPUT

```

[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[2]: data = "Dataset/titanic.csv"
df = pd.read_csv(data)

[3]: df.head()

[3]:
PassengerId  Survived    Name  Pclass  Sex  Age  SibSp  Parch    Ticket   Fare  Cabin Embarked
0           1         1  Braund, Mr. Owen Harris    3   male  22.0    1    0      A/5 21171   7.2500   NaN    S
1           2         1  Cumings, Mrs. John Bradley    1  female  38.0    1    0      PC 17599  53.1000   C85    C
2           3         1  Heikkinen, Miss. Laina    3  female  26.0    0    0  STON/O2. 3101282   7.9250   NaN    S
3           4         1  Futrelle, Mrs. Jacques Heath    1  female  35.0    1    0    113803  53.1000  C123    S
4           5         0  Allen, Mr. William Henry    3   male  35.0    0    0    373450   8.0500   NaN    S

[4]: df.tail()

[4]:
PassengerId  Survived    Name  Pclass  Sex  Age  SibSp  Parch    Ticket   Fare  Cabin Embarked
46           0         0  Rogers, Mr. William John    3   male  NaN    0    0    S.C./A.4. 23367   8.0500   NaN    S
47           0         0  Rice, Mrs. William (Margaret Norton)    3  female  NaN    0    0    382652  29.1250   NaN    Q
48           1         0  Porter, Mr. Henry    1   male  45.0    1    0    110465  83.4750   C66    S
49           0         0  Skoog, Master. Karl Thorsten    3   male   9.0    1    1    347742  11.1333   NaN    S
50           1         0  Arnold-Forster, Mrs. Josef (Josefina Forster)    3  female  18.0    1    0    349237  17.8000   NaN    S

[5]: df.sample(5)

[5]:
PassengerId  Survived    Name  Pclass  Sex  Age  SibSp  Parch    Ticket   Fare  Cabin Embarked
39           0         0  McMahon, Mr. Martin    3   male  NaN    0    0    370374   7.7500   NaN    Q
34           1         0  Graham, Miss. Margaret Edith    1  female  58.0    0    0    PC 17582  153.4625  C125    S
19           0         0  Masselmani, Mrs. Fatima    3  female  NaN    0    0    2649    7.2250   NaN    C
42           0         0  Carter, Mr. William Ernest    2   male  32.0    0    0    244252  13.0000   NaN    S
29           0         0  Todoroff, Mr. Lallo    3   male  NaN    0    0    349216   7.8958   NaN    S

```

```

[5]: df.sample(5)

[5]:
PassengerId  Survived    Name  Pclass  Sex  Age  SibSp  Parch    Ticket   Fare  Cabin Embarked
39           0         0  McMahon, Mr. Martin    3   male  NaN    0    0    370374   7.7500   NaN    Q
34           1         0  Graham, Miss. Margaret Edith    1  female  58.0    0    0    PC 17582  153.4625  C125    S
19           0         0  Masselmani, Mrs. Fatima    3  female  NaN    0    0    2649    7.2250   NaN    C
42           0         0  Carter, Mr. William Ernest    2   male  32.0    0    0    244252  13.0000   NaN    S
29           0         0  Todoroff, Mr. Lallo    3   male  NaN    0    0    349216   7.8958   NaN    S

[6]: print("Rows and Columns:", df.shape)
Rows and Columns: (51, 12)

[7]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51 entries, 0 to 50
Data columns (total 12 columns):
#   Column  Non-Null Count  Dtype
---  --
0  PassengerId  51 non-null      int64
1  Survived    51 non-null      int64
2  Name        51 non-null      object
3  Pclass      51 non-null      int64
4  Sex         51 non-null      object
5  Age         39 non-null      float64
6  SibSp       51 non-null      int64
7  Parch       51 non-null      int64
8  Ticket      51 non-null      object
9  Fare        51 non-null      float64
10 Cabin     12 non-null      object
11 Embarked   51 non-null      object
dtypes: float64(2), int64(5), object(5)
memory usage: 4.9+ KB

[8]: df.describe()

[8]:
PassengerId  Survived    Pclass    Age    SibSp    Parch    Fare
count  51.000000  51.000000  51.000000  39.000000  51.000000  51.000000  51.000000
mean    25.607843    0.509804    2.392157  26.867436    0.549020    0.470588    30.392565

```

Exercise 1 Last Checkpoint: 11 days ago

```
[9]: df.size
[9]: 812

[10]: df.columns

[10]: Index(['PassengerId', 'Survived', 'Name', 'Pclass', 'Sex', 'Age', 'SibSp',
         'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
        dtype='object')

[11]: df.isnull().sum()

[11]: PassengerId    0
      Survived      0
      Name         0
      Pclass       0
      Sex          0
      Age         12
      SibSp        0
      Parch        0
      Ticket       0
      Fare         0
      Cabin       14
      Embarked     0
      dtype: int64

[12]: missing_percentage = (df.isnull().sum() / len(df)) * 100
      print(missing_percentage, sort_values(ascending=False))

[12]: Cabin       1.711958
      Age         1.477708
      PassengerId  0.001478
      Survived    0.001478
      Name        0.001478
      Pclass      0.001478
      Sex         0.001478
      SibSp       0.001478
      Parch       0.001478
      Ticket      0.001478
      Fare        0.001478
      Embarked    0.001478
      dtype: float64

[13]: plt.figure(figsize=(10, 6))
sns.histmap
```

31°C Mostly clear

Search

ENG IN 97% 11:27 PM 10/8/2026

Exercise 1 Last Checkpoint: 11 days ago

```
[14]: # Fill Age missing values
      df["Age"] = df["Age"].fillna(df["Age"].median())

      # Fill Embarked missing values
      df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

      # Fill Cabin missing values
      df["Cabin"] = df["Cabin"].fillna("Unknown")

      print(df.isnull().sum())

      PassengerId    0
      Survived      0
      Name         0
      Pclass       0
      Sex          0
      Age          0
      SibSp        0
      Parch        0
      Ticket       0
      Fare         0
      Cabin         0
      Embarked     0
      dtype: int64

[15]: print("Number of duplicate rows:", df.duplicated().sum())

      Number of duplicate rows: 0

[16]: df = df.drop_duplicates()

      print("Shape after removing duplicates:", df.shape)

      Shape after removing duplicates: (51, 12)

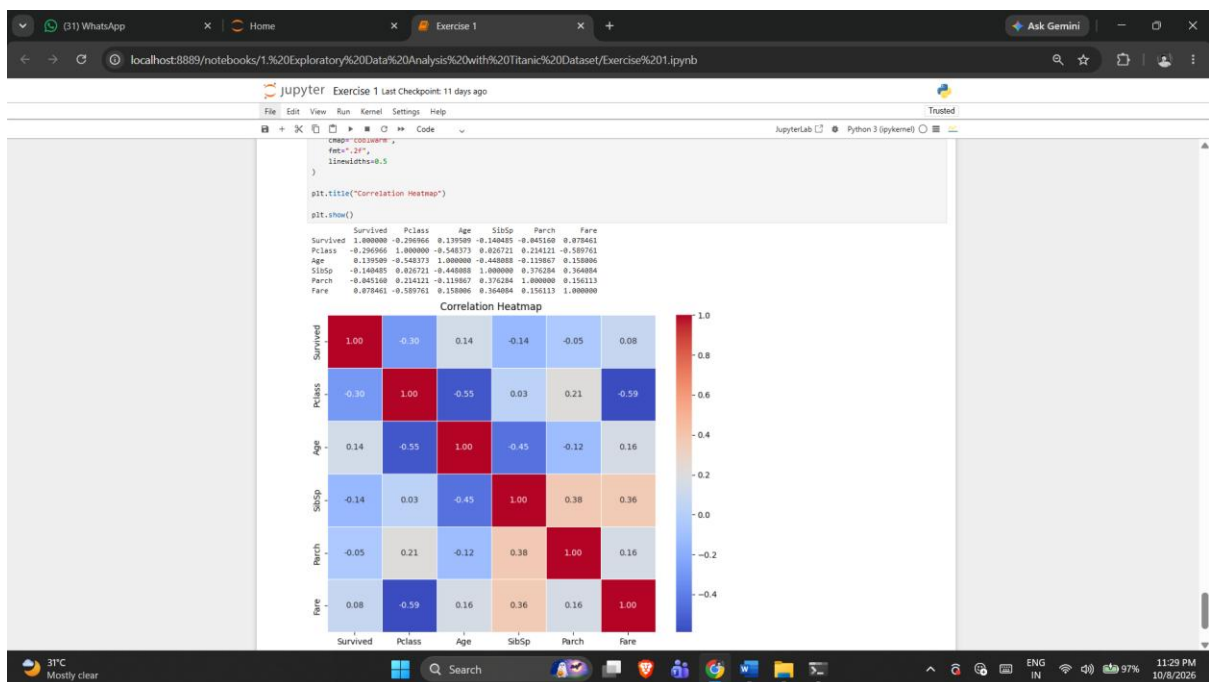
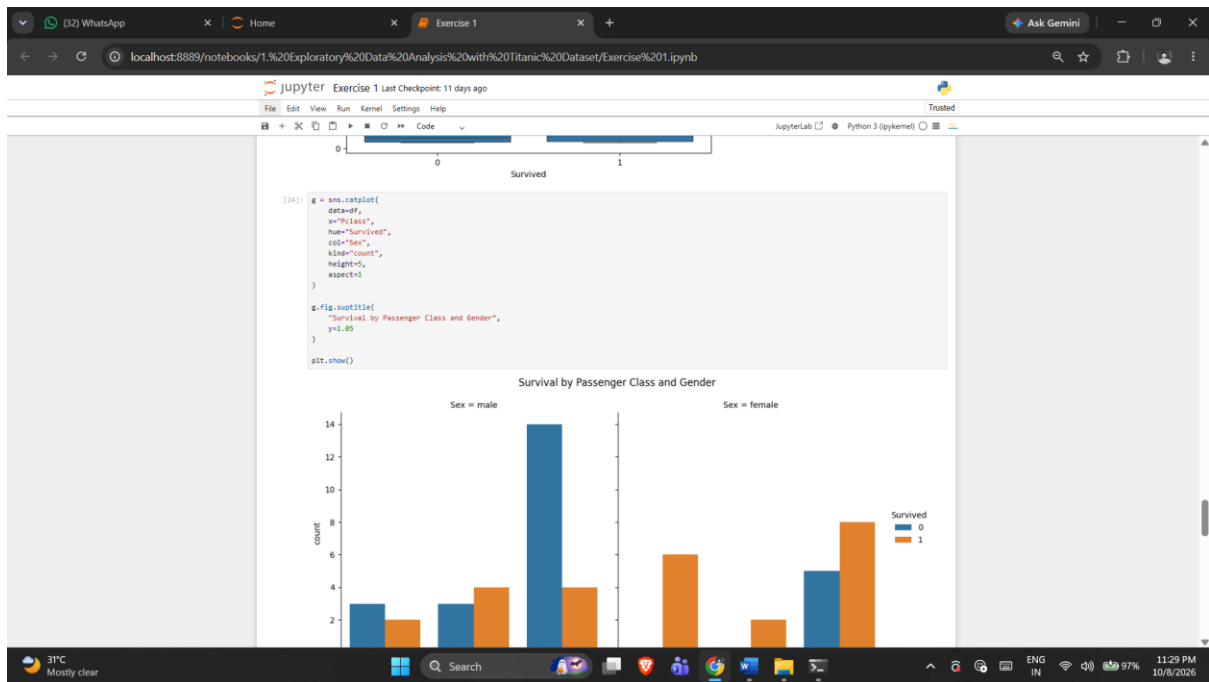
[17]: plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Survived")

plt.title("Passenger Survival Count")
plt.xlabel("Survived")
plt.ylabel("Number of Passengers")
plt.show()
```

31°C Mostly clear

Search

ENG IN 97% 11:28 PM 10/8/2026



5. RESULT

Thus, the Exploratory Data Analysis with Titanic Dataset practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -2

Simple Linear Regression

1. AIM

To implement and demonstrate Simple Linear Regression using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load the regression data.
2. Select the independent and dependent variables.
3. Split the data into training and testing sets.
4. Train Linear Regression.
5. Generate predictions.
6. Calculate the notebook's evaluation metrics.
7. Plot the fitted regression relationship.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = "Dataset/Salary.csv"
df = pd.read_csv(data)
df = df[['YearsExperience', 'Salary']]

df

df.size

df.shape

df.isnull().sum()

X = df[['YearsExperience']] # Independent Variable
y = df['Salary']           # Dependent Variable

model = LinearRegression()

model.fit(x1,y1)

x1,x2,y1,y2 = train_test_split(X,y,random_state= 42,test_size=0.2)
```

```

y_pred = model.predict(x2)

print("Actual Values:")
print(y2.values)

print("Predicted Values:")
print(y_pred)

print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

YearsExperience = [[2.1]]
predicted_salary = model.predict(YearsExperience)

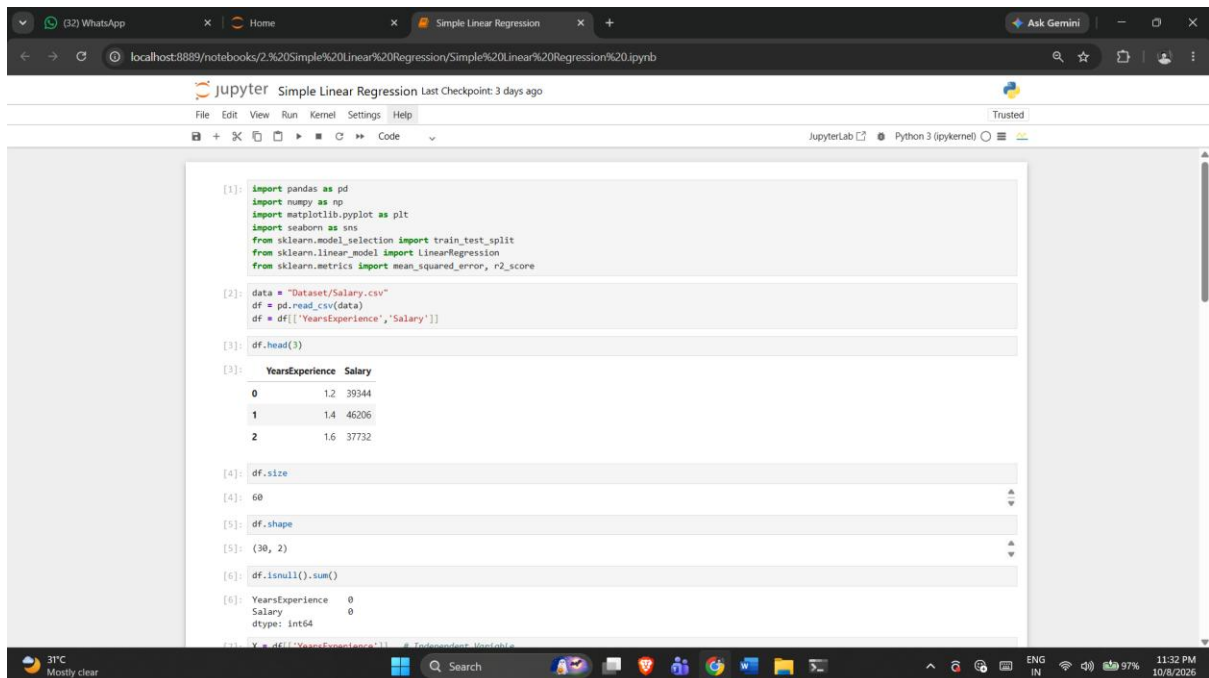
print("Predicted salary for 2.1 years:", predicted_salary[0])

plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')

plt.xlabel("Experience")
plt.ylabel("salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()

```

4. OUTPUT



The screenshot displays a JupyterLab environment with a notebook titled "Simple Linear Regression". The notebook is running on a local host (localhost:8889). The code in the notebook includes imports for pandas, numpy, matplotlib, seaborn, and sklearn. It loads a dataset from "Dataset/Salary.csv", preprocesses it by dropping missing values and splitting into features and targets, and then trains a linear regression model. The output shows the first three rows of the dataset and the model's predictions.

```

[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

[2]: data = "Dataset/Salary.csv"
df = pd.read_csv(data)
df = df[['YearsExperience', 'Salary']]

[3]: df.head(3)

[3]:
  YearsExperience  Salary
0              1.2  39344
1              1.4  46206
2              1.6  37732

[4]: df.size

[4]: 60

[5]: df.shape

[5]: (30, 2)

[6]: df.isnull().sum()

[6]:
YearsExperience    0
Salary            0
dtype: int64

```

Simple Linear Regression Last Checkpoint: 3 days ago

```
[7]: X = df[['YearsExperience']] # Independent Variable
     y = df['Salary'] # Dependent Variable

[10]: model = LinearRegression()
      x1,x2,y1,y2 = train_test_split(X,y, test_size= 0.2, random_state= 42)

[11]: model.fit(x1,y1)

[11]: LinearRegression
      LinearRegression()

[12]: y_pred = model.predict(x2)

      print("Actual Values:")
      print(y2.values)

      print("Predicted Values:")
      print(y_pred)

      Actual Values:
      [112636  67939 113813  83089  64446  57190]
      Predicted Values:
      [115791.21011287  71499.27809463 102597.86866063  75268.80422384
       55478.79204548  60190.69970699]

[14]: print("Coefficient (Slope):", model.coef_[0])
      print("Intercept:", model.intercept_)
      print("Mean Squared Error:", mean_squared_error(y2, y_pred))
      print("R2 Score:", r2_score(y2, y_pred))

      Coefficient (Slope): 9423.815323030973
      Intercept: 24380.201479473726
      Mean Squared Error: 418330096.855908364
      R2 Score: 0.9024461774180498
```

```
      print("Actual Values:")
      print(y2.values)

      print("Predicted Values:")
      print(y_pred)

      Actual Values:
      [112636  67939 113813  83089  64446  57190]
      Predicted Values:
      [115791.21011287  71499.27809463 102597.86866063  75268.80422384
       55478.79204548  60190.69970699]

[14]: print("Coefficient (Slope):", model.coef_[0])
      print("Intercept:", model.intercept_)
      print("Mean Squared Error:", mean_squared_error(y2, y_pred))
      print("R2 Score:", r2_score(y2, y_pred))

      Coefficient (Slope): 9423.815323030973
      Intercept: 24380.201479473726
      Mean Squared Error: 418330096.855908364
      R2 Score: 0.9024461774180498

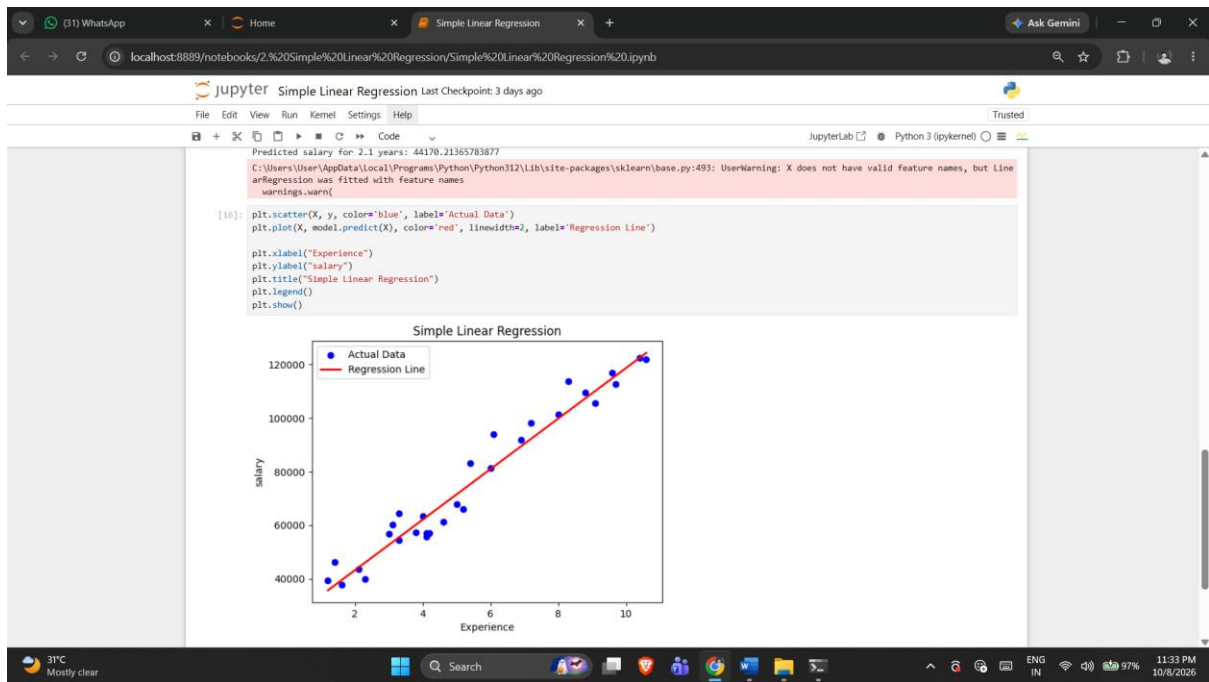
[15]: YearsExperience = [[2.1]]
      predicted_salary = model.predict(YearsExperience)

      print("Predicted salary for 2.1 years:", predicted_salary[0])

      Predicted salary for 2.1 years: 44170.21365783877
      C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
        warnings.warn(

[16]: plt.scatter(X, y, color='blue', label='Actual Data')
      plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')

      plt.xlabel("Experience")
      plt.ylabel("Salary")
      plt.title("Simple Linear Regression")
      plt.legend()
      plt.show()
```



5. RESULT

Thus, the Simple Linear Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -3

Multiple Linear Regression

1. AIM

To implement and demonstrate Multiple Linear Regression using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load the regression dataset.
2. Select the target and multiple predictor variables.
3. Split the data into training and testing sets.
4. Train the multiple linear regression model.
5. Generate predictions.
6. Calculate the regression metrics implemented in the notebook.
7. Display the notebook's visual analysis.

3. PROGRAM

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)

data = 'Dataset/Housing.csv'

df= pd.read_csv(data)

df.head()

df.info()

le = LabelEncoder()
for col in df.select_dtypes(include="object").columns:
    df[col] = le.fit_transform(df[col])

X = df.drop("price", axis=1)
y = df["price"]
```

```

x1, x2, y1, y2 = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(x1, y1)

y_pred = model.predict(x2)
y_pred

mae = mean_absolute_error(y2, y_pred)
mse = mean_squared_error(y2, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y2, y_pred)
mape = mean_absolute_percentage_error(y2, y_pred)
evs = explained_variance_score(y2, y_pred)
maxerr = max_error(y2, y_pred)

n = x2.shape[0]      # Number of observations
p = x2.shape[1]      # Number of predictors

adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

print("\n===== Regression Evaluation =====")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R2 Score: {r2:.4f}")
print(f"Adjusted R2 Score: {adj_r2:.4f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.4f}")
print(f"Explained Variance Score: {evs:.4f}")
print(f"Maximum Error: {maxerr:.2f}")

print("\nIntercept:")
print(model.intercept_)

# In[21]:

print("\nCoefficients:")
coef = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
print(coef)

new_house = [[5000, 3, 2, 2, 1, 1, 0, 1, 0, 1, 0, 1]]

prediction = model.predict(new_house)

print("\nPredicted House Price:", prediction[0])

import matplotlib.pyplot as plt
plt.figure(figsize=(6, 6))
plt.scatter(y2, y_pred)
plt.plot([y2.min(), y2.max()],
         [y2.min(), y2.max()],
         'r--')

```

```
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Values")
plt.show()
```

4. OUTPUT

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)
```

```
[2]: data = "Dataset/housing.csv"
df = pd.read_csv(data)
df.head(3)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   price               545 non-null   int64   
 1   area                545 non-null   int64   
 2   bedrooms            545 non-null   int64   
 3   bathrooms            545 non-null   int64   
 4   stories              545 non-null   int64   
 5   mainroad            545 non-null   object  
 6   guestroom           545 non-null   object  
 7   basement             545 non-null   object  
 8   hotwaterheating     545 non-null   object  
 9   airconditioning     545 non-null   object  
10   parking              545 non-null   int64   
11   prefarea             545 non-null   object  
12   furnishingstatus    545 non-null   object  
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   price               545 non-null   int64   
 1   area                545 non-null   int64   
 2   bedrooms            545 non-null   int64   
 3   bathrooms            545 non-null   int64   
 4   stories              545 non-null   int64   
 5   mainroad            545 non-null   object  
 6   guestroom           545 non-null   object  
 7   basement             545 non-null   object  
 8   hotwaterheating     545 non-null   object  
 9   airconditioning     545 non-null   object  
10   parking              545 non-null   int64   
11   prefarea             545 non-null   object  
12   furnishingstatus    545 non-null   object  
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

```
[4]: le = LabelEncoder()
for col in df.select_dtypes(include="object").columns:
    df[col] = le.fit_transform(df[col])
```

```
[5]: X = df.drop("price", axis=1)
y = df["price"]
```

```
[6]: x1, x2, y1, y2 = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[7]: model = LinearRegression()
model.fit(x1, y1)
```

```
[7]: LinearRegression
LinearRegression()
```

```
[8]: y_pred = model.predict(x2)
```

```
localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb

jupyter Multiple Linear Regression Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[0]: y_pred = model.predict(x2)
     y_pred

[0]: array([[5203691.70963178, 7257004.02115476, 3062828.59668172,
4559591.65374424, 3332932.30559782, 3563080.67918997,
5645466.31219972, 6413979.66873635, 2755831.54819 ,
2668938.66875229, 9570600.29915353, 2827431.50860865,
3195686.2583409 , 3352263.99438471, 3713879.49996131,
5301088.24435749, 2987920.2666968 , 4818799.8212371 ,
4383871.70489929, 3525092.18938647, 5796259.50860813,
5840000.70299301, 2760214.688641 , 4782590.14920607,
5204755.73895206, 7515542.71619025, 3254681.68956382,
5236164.45964444, 8178523.16820284, 3434166.15675649,
6443921.58767581, 3346084.77919184, 6742324.74084133,
4154936.84888665, 3589152.47491253, 5788125.92515323,
4768370.18154876, 4391684.04193171, 3217657.04549935,
4638196.61928878, 4522160.27786714, 3541284.86127245,
7238136.11941171, 4021515.6892614, 3701978.76822757,
4296879.55563898, 6785004.0206061 , 3993466.52296896,
3798185.05328859, 3451821.5624289 , 7293996.86867461,
2832985.26972678, 4378698.14769931, 4468002.85418353,
3714623.89765883, 2718466.9028278 , 7534449.64982626,
2950437.1495281 , 4194596.28592921, 2795827.77318101,
4801788.72577028, 3618230.29875972, 5091685.77019752,
4248122.70647652, 4729964.20281946, 4620982.9363411 ,
7215846.76983926, 3485106.1912682, 5933820.14349913,
6236580.98095548, 4809923.61015999, 5130920.22938947,
4527282.87392583, 8006869.07208366, 3543927.63713716,
5463086.05201817, 3902441.28505163, 4661003.61495886,
4825370.89312133, 4716803.47099967, 7754072.71344458,
4008573.23235629, 6580977.91345038, 5386796.00909809,
2785343.0092764 , 6836356.36771846, 2638553.49378047,
3634231.99215348, 8004072.41154001, 8048399.75418244,
3227573.69687309, 5035167.4198354 , 3685772.48317455,
3817767.91420862, 7631673.34074558, 4775106.62670537,
5093491.44442931, 6447437.14797692, 4943449.6813564 ,
5943929.14295426, 3810460.26405425, 6457426.32302761,
3701491.23218176, 5960492.47343074, 4933137.14539295,
4377133.13624287, 7009335.02696999, 6351468.97128313,
6409340.41089671])
```

```
localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb

jupyter Multiple Linear Regression Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

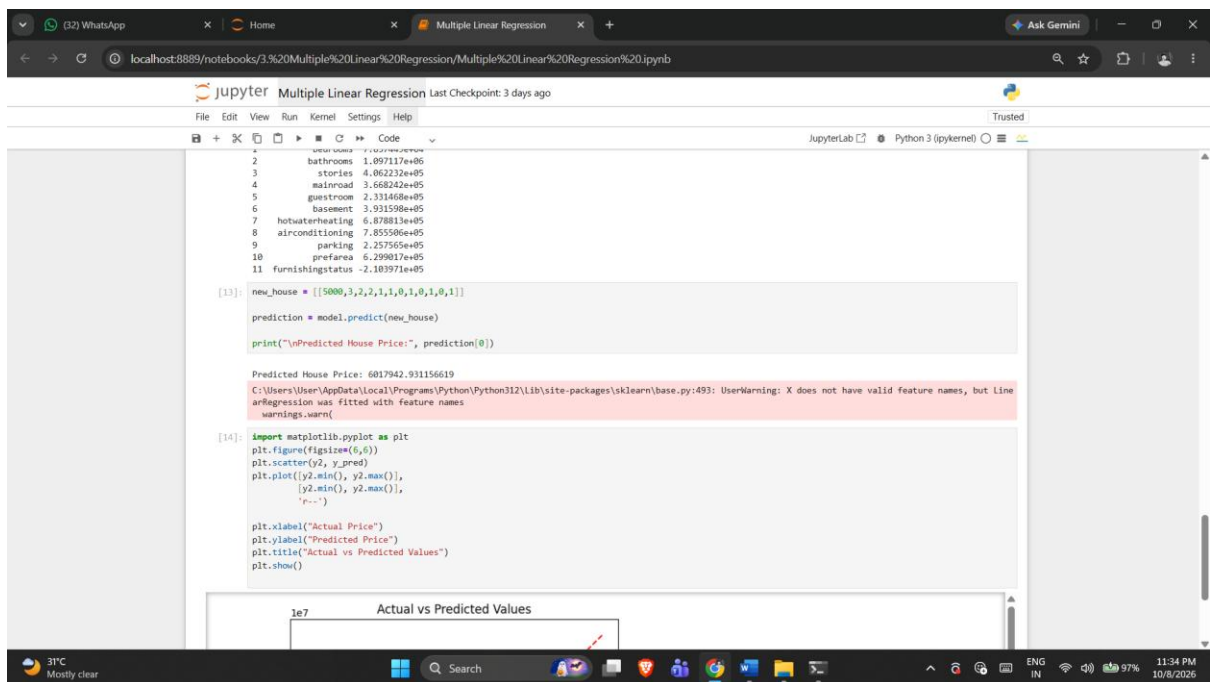
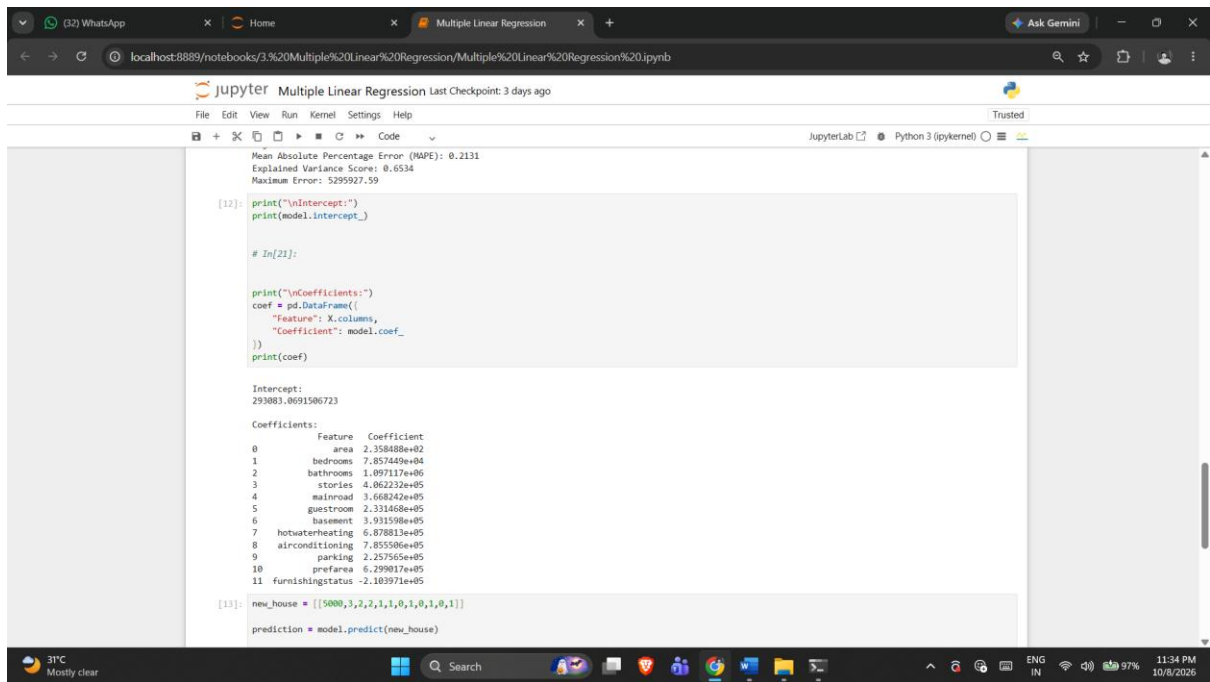
[9]: mae = mean_absolute_error(y2, y_pred)
     mse = mean_squared_error(y2, y_pred)
     rmse = np.sqrt(mse)
     r2 = r2_score(y2, y_pred)
     mape = mean_absolute_percentage_error(y2, y_pred)
     evs = explained_variance_score(y2, y_pred)
     maxerr = max_error(y2, y_pred)

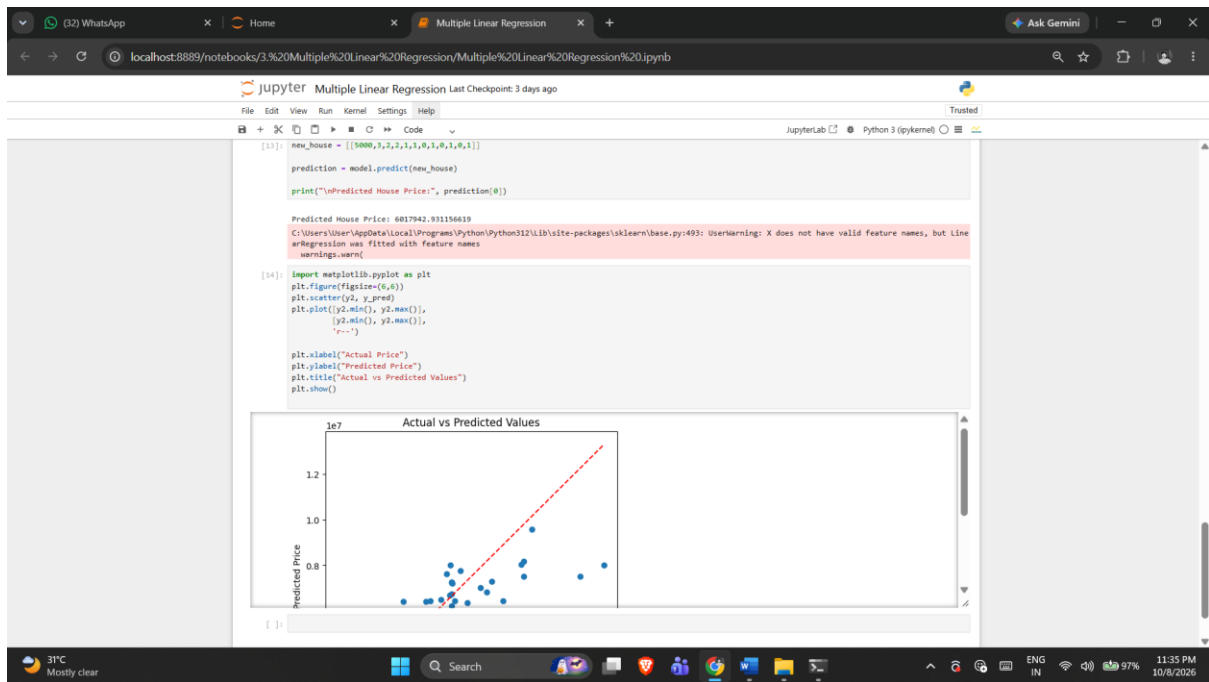
[10]: n = x2.shape[0] # Number of observations
      p = x2.shape[1] # Number of predictors
      adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

[11]: print("\n===== Regression Evaluation =====")
      print(f"Mean Absolute Error (MAE): (mae:.2f)")
      print(f"Mean Squared Error (MSE): (mse:.2f)")
      print(f"Root Mean Squared Error (RMSE): (rmse:.2f)")
      print(f"R² Score: (r2:.4f)")
      print(f"Adjusted R² Score: (adj_r2:.4f)")
      print(f"Mean Absolute Percentage Error (MAPE): (mape:.4f)")
      print(f"Explained Variance Score: (evs:.4f)")
      print(f"Maximum Error: (maxerr:.2f)")

      ----- Regression Evaluation -----
      Mean Absolute Error (MAE): 979679.69
      Mean Squared Error (MSE): 1771751116594.03
      Root Mean Squared Error (RMSE): 1331071.42
      R² Score: 0.6495
      Adjusted R² Score: 0.6057
      Mean Absolute Percentage Error (MAPE): 0.2131
      Explained Variance Score: 0.6534
      Maximum Error: 5295927.59

[12]: print("\nIntercept:")
      print(model.intercept_)
```





5. RESULT

Thus, the Multiple Linear Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -4

Polynomial Regression

1. AIM

To implement degree-2 polynomial regression, evaluate its predictions with the notebook's regression metrics, and visualize the fitted curve.

2. ALGORITHM

1. Load and inspect the fish dataset.
2. Encode the Species feature.
3. Separate Weight as target.
4. Split into training and testing sets.
5. Generate degree-2 polynomial features.
6. Train Linear Regression on transformed features.
7. Calculate all metrics implemented in the notebook.
8. Plot the polynomial relationship for Length3 and Weight.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)

data = 'Dataset/fish.csv'
df = pd.read_csv(data)

df.head()

df.info()

df.shape

df = pd.get_dummies(df, columns=['Species'], drop_first=True)
```

```

X = df.drop("Weight", axis=1)
y = df["Weight"]

x1, x2, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(x1)
X_test_poly = poly.transform(x2)

model = LinearRegression()

model.fit(X_train_poly, y_train)

# Prediction
y_pred = model.predict(X_test_poly)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)

# Adjusted R2
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1

adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

print("\n===== Evaluation Metrics =====")

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
print("Adjusted R2 :", adj_r2)
print("MAPE :", mape)
print("Explained Variance :", evs)
print("Maximum Error :", maxerr)

X = df[['Length3']]
y = df['Weight']
poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)

X_grid = np.arange(
    X.min().values[0],

```

```

        X.max().values[0],
        0.1
    ).reshape(-1,1)

plt.figure(figsize=(8,6))

plt.scatter(X, y, color="blue", label="Actual Data")

plt.plot(
    X_grid,
    model.predict(poly.transform(X_grid)),
    color="red",
    linewidth=3,
    label="Polynomial Curve"
)

plt.xlabel("Length3")
plt.ylabel("Weight")
plt.title("Polynomial Regression")
plt.legend()
plt.show()

```

4. OUTPUT

The screenshot shows a Jupyter Notebook titled "Polynomial Regression" with the following code and output:

```

[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)

```

```

[2]: data = 'Dataset/fish.csv'
df = pd.read_csv(data)
df.head()

```

	Species	Weight	Length1	Length2	Length3	Height	Width
0	Bream	242.0	23.2	25.4	30.0	11.5200	4.0200
1	Bream	290.0	24.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	23.9	26.5	31.1	12.3778	4.6961
3	Bream	363.0	26.3	29.0	33.5	12.7300	4.4555
4	Bream	430.0	26.5	29.0	34.0	12.4440	5.1340

```

[3]: df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 159 entries, 0 to 158
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype  
---  --
 0   Species    159 non-null    object  
 1   Weight     159 non-null    float64 
 2   Length1    159 non-null    float64 
 3   Length2    159 non-null    float64 
 4   Length3    159 non-null    float64 
 5   Height     159 non-null    float64 
 6   Width      159 non-null    float64 
dtypes: object(1), float64(6)
memory usage: 13.5 KB

```

Polynomial Regression Last Checkpoint: 3 days ago

```
[4]: df.shape
[4]: (159, 7)

[5]: df = pd.get_dummies(df, columns=['Species'], drop_first=True)

[6]: X = df.drop("Weight", axis=1)
y = df["Weight"]

[7]: x1, x2, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

[8]: poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(x1)
X_test_poly = poly.transform(x2)

[9]: model = LinearRegression()
model.fit(X_train_poly, y_train)

# Prediction
y_pred = model.predict(X_test_poly)

[10]: mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)

[11]: # Adjusted R^2
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1
```

31°C Mostly clear 11:37 PM 10/8/2026

Polynomial Regression Last Checkpoint: 3 days ago

```
model.fit(X_train_poly, y_train)

# Prediction
y_pred = model.predict(X_test_poly)

[10]: mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)

[11]: # Adjusted R^2
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1
adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

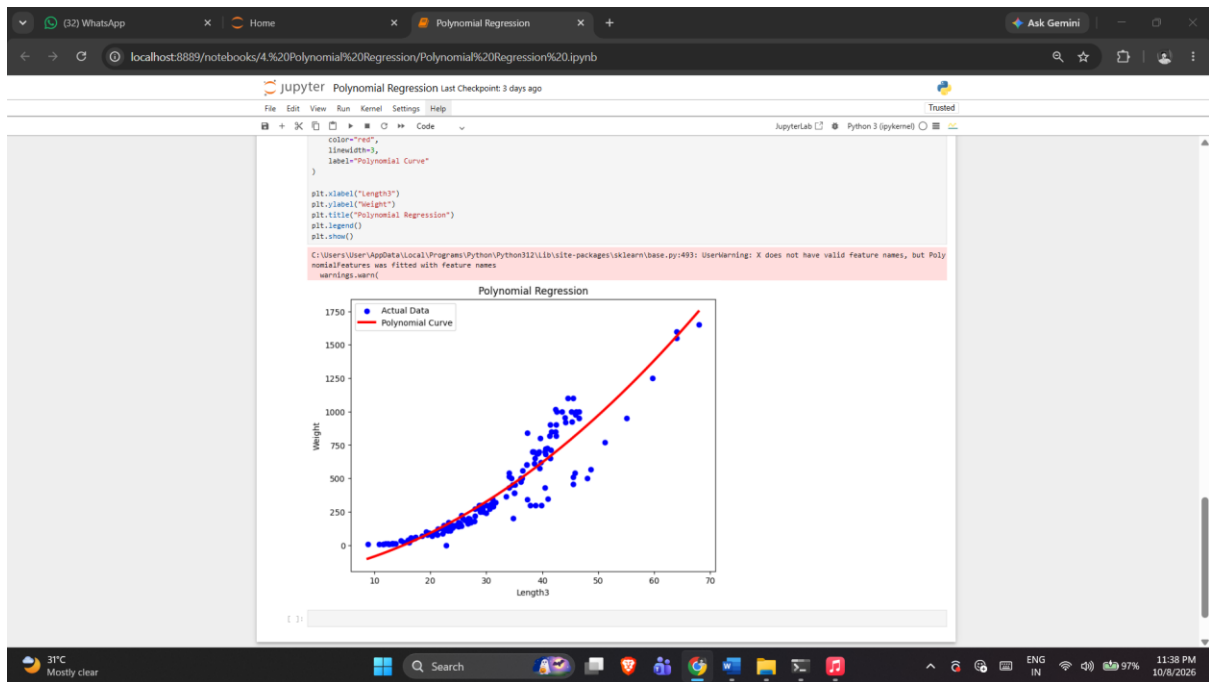
print("***** Evaluation Metrics *****")

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
print("Adjusted R2 :", adj_r2)
print("MAPE :", mape)
print("Explained Variance :", evs)
print("Maximum Error :", maxerr)

***** Evaluation Metrics *****
MAE : 53.639221496582836
MSE : 6047.62217520895
RMSE : 82.79030874483
R2 Score : 0.9518584328737167
Adjusted R2 : 1.0324432300198865
MAPE : 0.2586239020683204
Explained Variance : 0.9545323484665824
Maximum Error : 290.5704345703125

[12]: X = df[["Length3"]]
y = df["Weight"]
poly = PolynomialFeatures(degree=2)
```

31°C Mostly clear 11:37 PM 10/8/2026



5. RESULT

Thus, the Polynomial Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -5

Regularization Techniques

1. AIM

To implement and demonstrate Regularization Techniques using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load and prepare the regression data.
2. Separate predictors and target.
3. Split the data.
4. Train the regularization models implemented in the notebook.
5. Generate predictions.
6. Compare models using the notebook's evaluation metrics.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import r2_score

data = 'Dataset/Housing.csv'
df = pd.read_csv(data)
df.head()

df = pd.get_dummies(df, drop_first=True)

df.head()

# In[5]:

X = df.drop("price", axis=1)
y = df["price"]

# In[6]:

X_train,X_test,y_train,y_test = train_test_split(
```

```
X,  
Y,  
test_size=0.2,  
random_state=42  
)
```

```
# In[7]:
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

```
# In[80]:
```

```
poly = PolynomialFeatures(  
    degree=4,  
    include_bias=False  
)
```

```
X_train_poly = poly.fit_transform(X_train_scaled)  
X_test_poly = poly.transform(X_test_scaled)
```

```
print("Original Features :", X_train.shape[1])  
print("Polynomial Features :", X_train_poly.shape[1])
```

```
# In[81]:
```

```
linear = LinearRegression()
```

```
linear.fit(X_train_poly, y_train)
```

```
train_pred = linear.predict(X_train_poly)  
test_pred = linear.predict(X_test_poly)
```

```
train_r2 = r2_score(y_train, train_pred)  
test_r2 = r2_score(y_test, test_pred)
```

```
print("LINEAR REGRESSION")  
print("Training R2 :", round(train_r2,4))  
print("Testing R2 :", round(test_r2,4))
```

```
# In[83]:
```

```
ridge = Ridge(alpha=100)
```

```

ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("RIDGE")
print("Training R2 :", round(ridge_train,4))
print("Testing R2 :", round(ridge_test,4))

# In[ ]:

lasso = Lasso(
    alpha=0.1,
    max_iter=50000
)

lasso.fit(X_train_poly, y_train)

train_pred = lasso.predict(X_train_poly)
test_pred = lasso.predict(X_test_poly)

lasso_train = r2_score(y_train, train_pred)
lasso_test = r2_score(y_test, test_pred)

print("LASSO")
print("Training R2 :", round(lasso_train,4))
print("Testing R2 :", round(lasso_test,4))

# In[77]:

elastic = ElasticNet(
    alpha=0.1,
    l1_ratio=0.5,
    max_iter=50000
)

elastic.fit(X_train_poly, y_train)

train_pred = elastic.predict(X_train_poly)
test_pred = elastic.predict(X_test_poly)

elastic_train = r2_score(y_train, train_pred)
elastic_test = r2_score(y_test, test_pred)

print("ELASTIC NET")
print("Training R2 :", round(elastic_train,4))
print("Testing R2 :", round(elastic_test,4))

```

```
# In[78]:
```

```
comparison = pd.DataFrame({

    "Model":[
        "Linear Regression",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],

    "Train R2":[
        train_r2,
        ridge_train,
        lasso_train,
        elastic_train
    ],

    "Test R2":[
        test_r2,
        ridge_test,
        lasso_test,
        elastic_test
    ]
})
```

```
print(comparison)
```

```
# In[79]:
```

```
# In[62]:
```

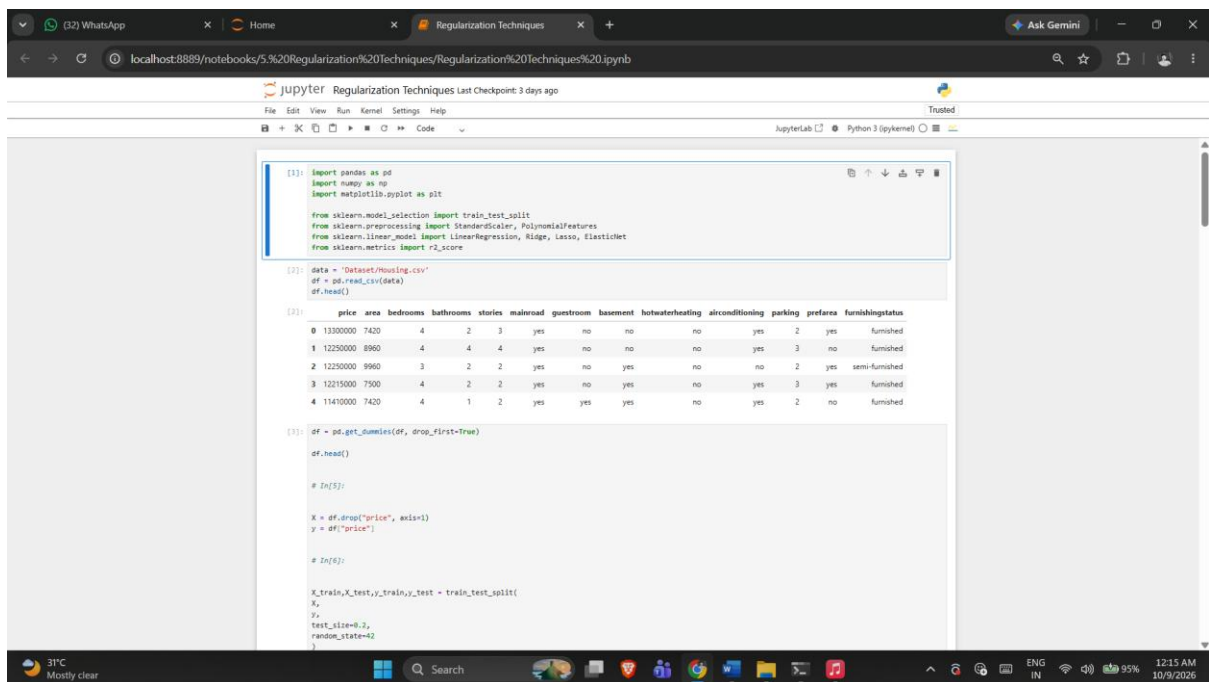
```
comparison["Difference"] = comparison["Train R2"] - comparison["Test R2"]
```

```
print(comparison)
```

```
comparison.plot(
    x="Model",
    y=["Train R2", "Test R2"],
    kind="bar",
    figsize=(10,6)
)
```

```
plt.title("Reducing Overfitting using Regularization")
plt.ylabel("R2 Score")
plt.xticks(rotation=10)
plt.grid(axis="y")
plt.show()
```

4. OUTPUT



```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import r2_score
```

```
[2]: data = 'Dataset/Housing.csv'
df = pd.read_csv(data)
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	yes	3	no	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12115000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished

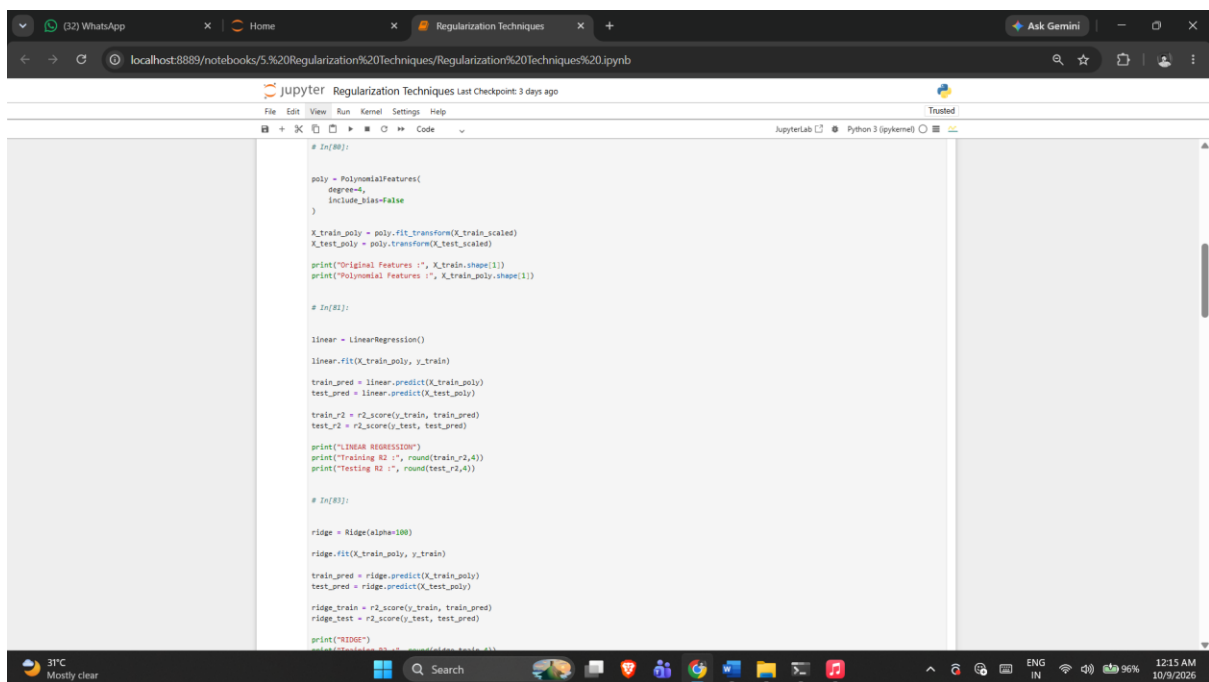
```
[3]: df = pd.get_dummies(df, drop_first=True)
df.head()

# In[5]:

X = df.drop("price", axis=1)
y = df["price"]

# In[6]:

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
```



```
# In[8]:

poly = PolynomialFeatures(
    degree=4,
    include_bias=False
)

X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

print("Original Features :", X_train.shape[1])
print("Polynomial Features :", X_train_poly.shape[1])

# In[9]:

linear = LinearRegression()
linear.fit(X_train_poly, y_train)

train_pred = linear.predict(X_train_poly)
test_pred = linear.predict(X_test_poly)

train_r2 = r2_score(y_train, train_pred)
test_r2 = r2_score(y_test, test_pred)

print("\nLINEAR REGRESSION")
print("Training R2 :", round(train_r2,4))
print("Testing R2 :", round(test_r2,4))

# In[10]:

ridge = Ridge(alpha=100)
ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("\nRIDGE")
```

Regularization Techniques

```
print("Training R2 :", round(train_r2,4))
print("Testing R2 :", round(test_r2,4))

# In[6]:

ridge = Ridge(alpha=100)
ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("RIDGE")
print("Training R2 :", round(ridge_train,4))
print("Testing R2 :", round(ridge_test,4))

# In[7]:

lasso = Lasso(
    alpha=0.1,
    max_iter=10000
)
lasso.fit(X_train_poly, y_train)

train_pred = lasso.predict(X_train_poly)
test_pred = lasso.predict(X_test_poly)

lasso_train = r2_score(y_train, train_pred)
lasso_test = r2_score(y_test, test_pred)

print("LASSO")
print("Training R2 :", round(lasso_train,4))
print("Testing R2 :", round(lasso_test,4))

# In[8]:

elastic = ElasticNet(
    alpha=0.1,
    l1_ratio=0.5,
    max_iter=10000
)

elastic.fit(X_train_poly, y_train)
```

```
elastic.fit(X_train_poly, y_train)

train_pred = elastic.predict(X_train_poly)
test_pred = elastic.predict(X_test_poly)

elastic_train = r2_score(y_train, train_pred)
elastic_test = r2_score(y_test, test_pred)

print("ELASTIC NET")
print("Training R2 :", round(elastic_train,4))
print("Testing R2 :", round(elastic_test,4))

# In[9]:

comparison = pd.DataFrame({

    "Model": [
        "Linear Regression",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],

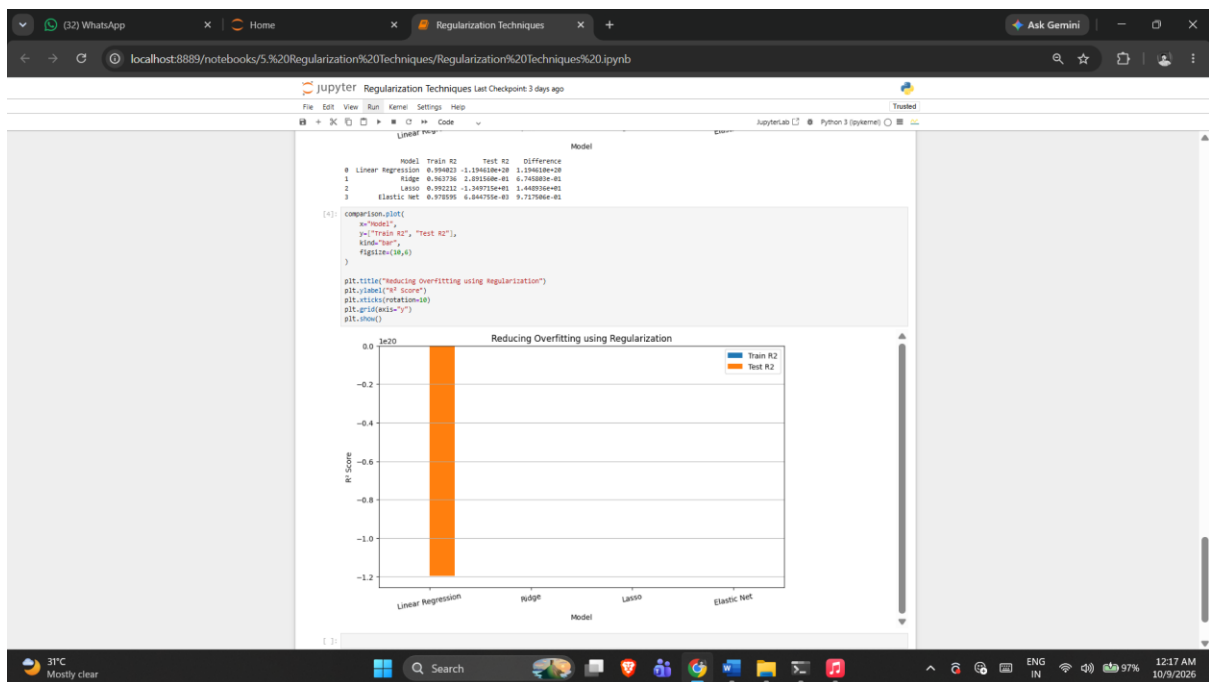
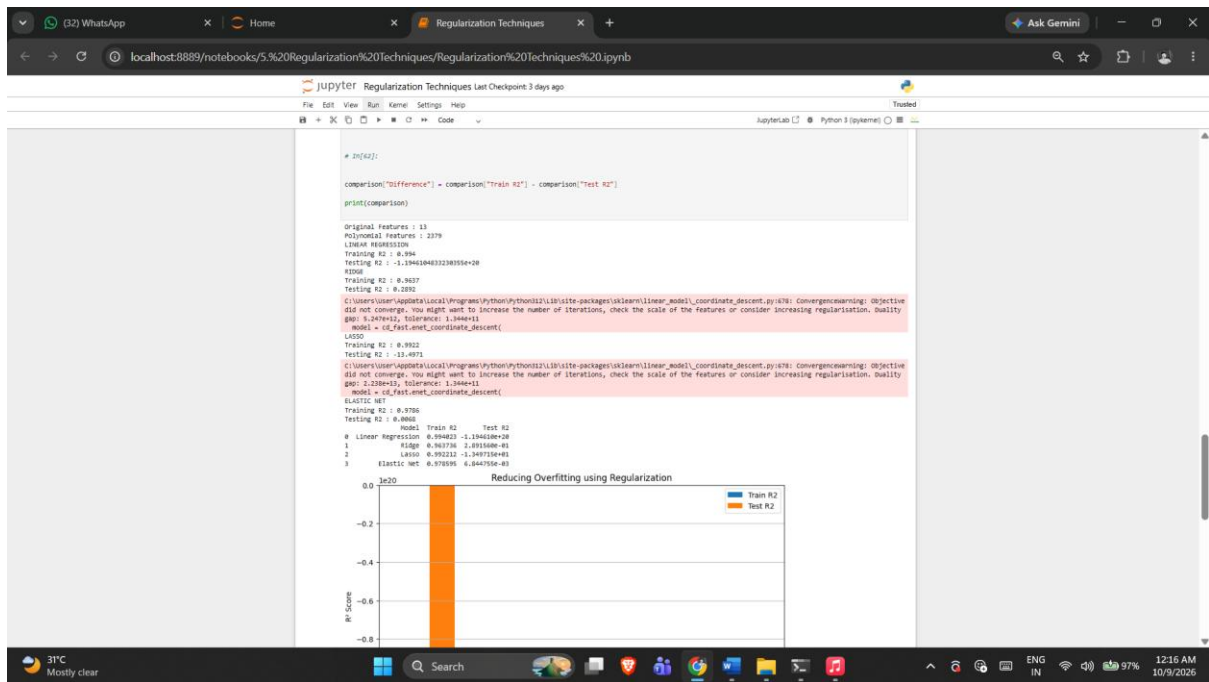
    "Train R2": [
        train_r2,
        ridge_train,
        lasso_train,
        elastic_train
    ],

    "Test R2": [
        test_r2,
        ridge_test,
        lasso_test,
        elastic_test
    ]
})

print(comparison)

# In[10]:

# In[11]:
```



5. RESULT

Thus, the Regularization Techniques practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -6

Logistic Regression

1. AIM

To implement Logistic Regression for binary heart disease classification and evaluate the model using the metrics implemented in the notebook.

2. ALGORITHM

1. Load the dataset and separate features and target.
2. Identify numerical and categorical features.
3. Create the preprocessing pipeline.
4. Split the data into training and testing sets.
5. Train the Logistic Regression model.
6. Predict the test set and calculate the notebook's metrics.
7. Display the model performance.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)

df = pd.read_csv("heart_disease_dataset.csv")

print("First 5 rows:")
print(df.head())
```

```

print("\nDataset Shape:")
print(df.shape)

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(y.name)

categorical_columns = X.select_dtypes(
    include=["object"]
).columns

numerical_columns = X.select_dtypes(
    exclude=["object"]
).columns

print("\nCategorical Columns:")
print(list(categorical_columns))

print("\Numerical Columns:")
print(list(numerical_columns))

imputer = SimpleImputer(strategy="most_frequent")

X[categorical_columns] = imputer.fit_transform(
    X[categorical_columns]
)

print("\nMissing values after imputation:")
print(X.isnull().sum())

label_encoders = {}

for column in categorical_columns:

    encoder = LabelEncoder()

    X[column] = encoder.fit_transform(
        X[column]
    )

    label_encoders[column] = encoder

print("\nData after Label Encoding:")
print(X.head())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,

```

```

        random_state=42,
        stratify=y
    )

print("\nTraining data:", X_train.shape)
print("Testing data :", X_test.shape)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# minmax_scaler = MinMaxScaler()

# X_train_scaled = minmax_scaler.fit_transform(X_train)

# X_test_scaled = minmax_scaler.transform(X_test)

model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
model.fit(
    X_train_scaled,
    y_train
)

print("\nLogistic Regression model trained successfully.")

y_pred = model.predict(X_test_scaled)
# Probability of class 1
y_probability = model.predict_proba(X_test_scaled)[:, 1]

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred
)

recall = recall_score(
    y_test,
    y_pred
)

f1 = f1_score(
    y_test,
    y_pred
)

print("\n=====")
print("CLASSIFICATION METRICS")

```

```

print("=====")

print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)

# =====
# 15. ROC Curve
# =====

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_probability
)

# =====
# 16. Calculate AUC
# =====

roc_auc = auc(
    fpr,
    tpr
)

print("\nROC-AUC Score:",
      round(roc_auc, 4))

# =====
# 17. Plot ROC Curve
# =====

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label="Logistic Regression (AUC = {:.3f})".format(
        roc_auc
    )
)

```

```

)

# Random classifier line
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

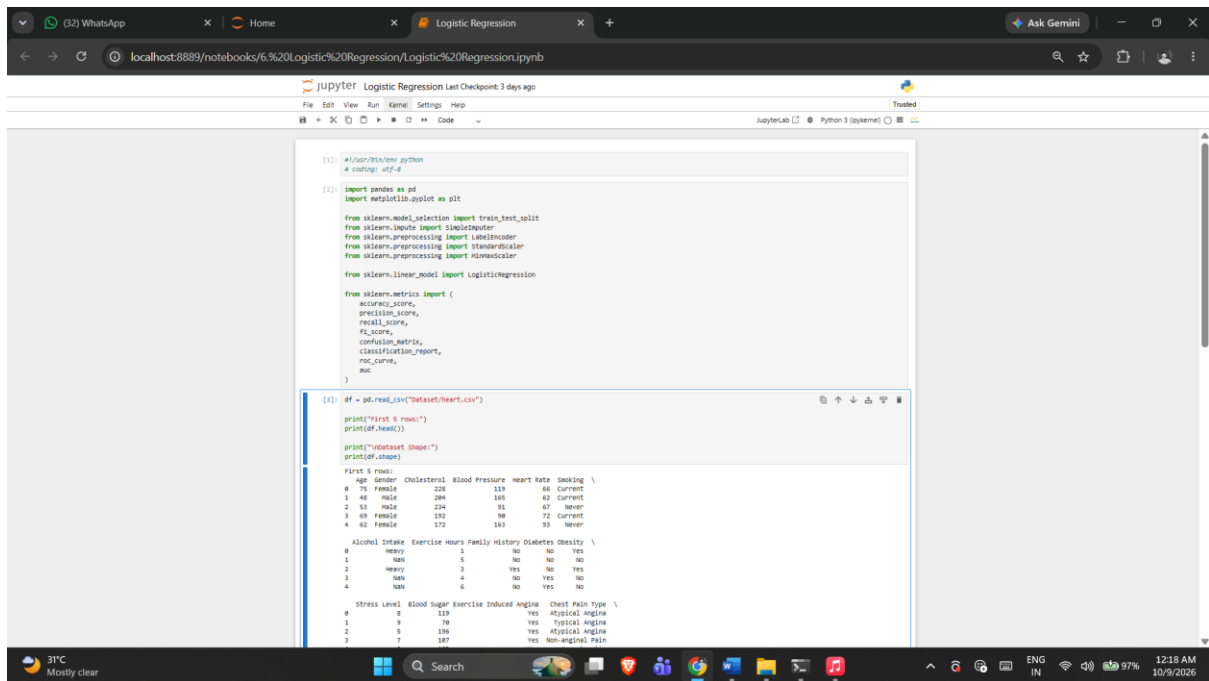
plt.title(
    "ROC Curve - Heart Disease Prediction"
)

plt.legend()

plt.grid()

```

4. OUTPUT



```

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)

[3]: df = pd.read_csv("Dataset/heart.csv")

print("First 5 rows")
print(df.head())

print("\ndataset shape:")
print(df.shape)

First 5 rows:
   Age  Gender  Cholesterol  Blood Pressure  heart rate  Smoking \
0  75  Female      228      119      64  Current
1  48  Male      284      145      62  Current
2  53  Male      234      91      67  Never
3  49  Female     182      98      72  Current
4  62  Female     172      163      93  Never

Alcohol Intake  Exercise hours  Family history  Diabetes  Obesity \
0  heavy      2      No  No  Yes
1  NaN      5      No  No  No
2  heavy      3      Yes  No  Yes
3  NaN      4      No  Yes  No
4  NaN      6      No  Yes  No

Stress Level  Blood sugar  Exercise induced Angina  Chest Pain Type \
0  5      119      Yes  Atypical Angina
1  9      78      Yes  Typical Angina
2  5      126      Yes  Atypical Angina
3  7      187      Yes  Non-anginal Pain

```

Logistic Regression

```
1 0
2 1
3 0
4 0

Dataset Shape:
(1000, 16)

[4]: x = df.drop("Heart Disease", axis=1)
     y = df["Heart Disease"]
     print("x features:")
     print(x.columns)

     print("y target:")
     print(y.name)

     Features:
     Index(['Age', 'Gender', 'Cholesterol', 'Blood Pressure', 'Heart Rate',
            'Smoking', 'Alcohol Intake', 'Exercise Hours', 'Family History',
            'Diabetes', 'Obesity', 'Stress Level', 'Blood Sugar',
            'Exercise Induced Angina', 'Chest Pain Type'],
           dtype=object)

     Target:
     Heart Disease

[5]: categorical_columns = x.select_dtypes(
     ).include("object").columns
     numerical_columns = x.select_dtypes(
     ).exclude("object").columns

     print("Categorical columns:")
     print(list(categorical_columns))

     print("Numerical columns:")
     print(list(numerical_columns))

     Categorical columns:
     ['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
     Numerical columns:
     ['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']

[6]: inputer = SimpleImputer(strategy="most_frequent")

     X[categorical_columns] = inputer.fit_transform(
     X[categorical_columns])

     print("Missing values after imputation:")
     print(X.isnull().sum())
```

Logistic Regression

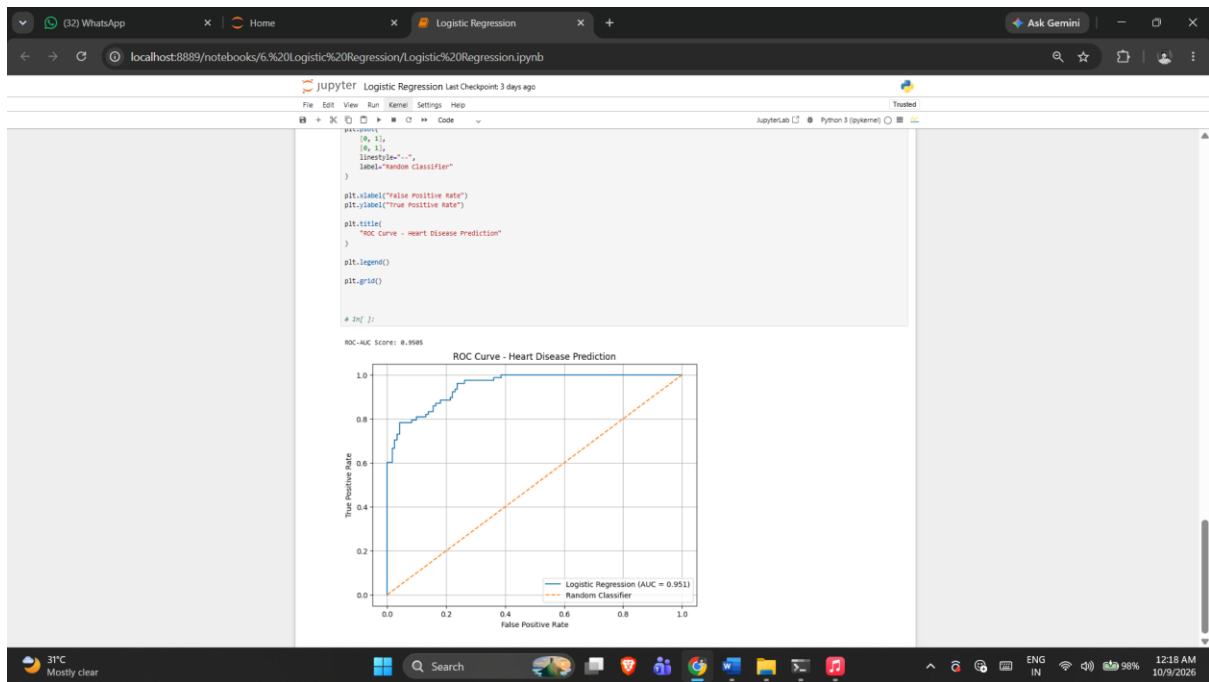
```
Missing values after imputation:
Age      0
Gender   0
Cholesterol  0
Blood Pressure  0
Heart Rate  0
Smoking   0
Alcohol Intake  0
Exercise Hours  0
Family History  0
Diabetes   0
Obesity   0
Stress Level  0
Blood Sugar  0
Exercise Induced Angina  0
Chest Pain Type  0
dtype: int64

[7]: label_encoders = {}

     for column in categorical_columns:
         encoder = LabelEncoder()
         X[column] = encoder.fit_transform(
         X[column])
         label_encoders[column] = encoder

     print("Vocabs after Label Encoding:")
     print(X.head())

     Data after Label Encoding:
     Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 0 220 110 66 0
1 40 1 204 105 62 0
2 53 1 234 92 67 2
3 69 0 192 90 72 0
4 62 0 172 105 63 2
```



5. RESULT

Thus, the Logistic Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -7

K-Nearest Neighbour

1. AIM

To implement K-Nearest Neighbour classification for heart disease prediction, compare multiple K values and select the best-performing K.

2. ALGORITHM

1. Load the dataset and separate features and target.
2. Identify numerical and categorical features.
3. Split the dataset with stratification.
4. Scale numerical features and encode categorical features.
5. Train KNN for the specified K values.
6. Calculate accuracy, precision, recall and F1 score.
7. Select the best K and evaluate the final model.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Records:")
```

```

print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nTarget Distribution:")
print(df["Heart Disease"].value_counts())

X = df.drop("Heart Disease", axis=1)

y = df["Heart Disease"]

numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

print("Numerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")
print(list(categorical_features))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data :", X_test.shape)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

K = 5
knn = KNeighborsClassifier(
    n_neighbors=5
)

knn.fit(
    X_train_processed,
    y_train

```

```

)

y_pred = knn.predict(
    X_test_processed
)

print("Predicted Values:")
print(y_pred[:20])

print(
    classification_report(
        y_test,
        y_pred
    )
)

k_values = [1, 3, 5, 7, 9, 11, 15, 21]

accuracy_values = []
precision_values = []
recall_values = []
f1_values = []

for k in k_values:

    knn = KNeighborsClassifier(
        n_neighbors=k
    )

    knn.fit(
        X_train_processed,
        y_train
    )

    y_pred = knn.predict(
        X_test_processed
    )

    accuracy_values.append(
        accuracy_score(
            y_test,
            y_pred
        )
    )

    precision_values.append(
        precision_score(
            y_test,
            y_pred
        )
    )

    recall_values.append(
        recall_score(
            y_test,
            y_pred
        )
    )

```

```

        fl_values.append(
            fl_score(
                y_test,
                y_pred
            )
        )

results = pd.DataFrame({
    "K": k_values,
    "Accuracy": accuracy_values,
    "Precision": precision_values,
    "Recall": recall_values,
    "F1 Score": fl_values
})

print(results)

plt.figure(figsize=(8, 5))

plt.plot(
    k_values,
    accuracy_values,
    marker="o"
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")

plt.title("Effect of K Value on KNN Accuracy")

plt.xticks(k_values)

plt.grid()

plt.show()

best_index = np.argmax(fl_values)

best_k = k_values[best_index]

print("Best K:", best_k)

print(
    "Best Accuracy:",
    accuracy_values[best_index]
)

print(
    "Best Precision:",
    precision_values[best_index]
)

print(
    "Best Recall:",
    recall_values[best_index]
)

```

```

print(
    "Best F1 Score:",
    fl_values[best_index]
)

final_knn = KNeighborsClassifier(
    n_neighbors=best_k
)

final_knn.fit(
    X_train_processed,
    y_train
)

final_pred = final_knn.predict(
    X_test_processed
)

final_accuracy = accuracy_score(
    y_test,
    final_pred
)

final_precision = precision_score(
    y_test,
    final_pred
)

final_recall = recall_score(
    y_test,
    final_pred
)

final_f1 = f1_score(
    y_test,
    final_pred
)

final_cm = confusion_matrix(
    y_test,
    final_pred
)

TN, FP, FN, TP = final_cm.ravel()

final_specificity = TN / (TN + FP)

final_probability = final_knn.predict_proba(
    X_test_processed
)[: , 1]

final_auc = roc_auc_score(
    y_test,
    final_probability
)

print("\n=====")
print("FINAL KNN MODEL")
print("=====")

```

```

print("Best K          :", best_k)
print("Accuracy        :", final_accuracy)
print("Precision        :", final_precision)
print("Recall           :", final_recall)
print("F1 Score         :", final_f1)
print("Specificity      :", final_specificity)
print("AUC              :", final_auc)

```

4. OUTPUT

```

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

[3]: data = 'dataset/heart.csv'
df = pd.read_csv(data)

print("Dataset shape:", df.shape)

print("\nFirst 5 Records:")
print(df.head())

Dataset shape: (1000, 16)

First 5 Records:
   age  gender  cholesterol  blood pressure  heart rate  smoking \
0    75    male          228             119           66    current
1    48    male          204             165           62    current
2    53    male          234             151           67    never
3    69    female        192             98           72    current
4    42    female        172             163           91    never

   alcohol intake  exercise hours  family history  diabetes  obesity \
0             heavy              1             no         no         yes
1             none              5             no         no         no
2             heavy              3             yes         no         yes
3             none              4             no         no         yes
4             none              6             no         no         no

   stress level  blood sugar  exercise induced angina  chest pain type \
0              0           119                    yes      atypical angina

```

```

[4]: print("\nDataset Information:")
print(df.info())

print("\nMissing values:")
print(df.isnull().sum())

print("\nTarget Distribution:")
print(df["heart_disease"].value_counts())

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1000 entries, 0 to 999
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   age                 1000 non-null   int64
 1   gender              1000 non-null   object
 2   cholesterol         1000 non-null   int64
 3   blood pressure      1000 non-null   int64
 4   heart rate          1000 non-null   int64
 5   smoking             1000 non-null   object
 6   alcohol intake      660 non-null    object
 7   exercise hours      1000 non-null   int64
 8   family history      1000 non-null   object
 9   diabetes            1000 non-null   object
10   obesity             1000 non-null   object
11   stress level        1000 non-null   int64
12   blood sugar         1000 non-null   int64
13   exercise induced angina 1000 non-null   object
14   chest pain type     1000 non-null   object
15   heart disease       1000 non-null   int64
dtypes: int64(1), object(15)
memory usage: 125.1+ KB
None

Missing values:
age                0
gender             0
cholesterol        0
blood pressure     0
heart rate         0
smoking            0
alcohol intake     340
exercise hours     0
family history     0
diabetes           0
obesity            0
stress level       0
blood sugar        0

```

localhost:8889/notebooks/7.%20K%20Nearest%20Neighbour/KNN%20Classifier.ipynb

Jupyter KNN Classifier Last Checkpoint: 3 days ago

```
target Distribution:
heart disease
# 406
1 392
Name: count, dtype: int64

[5]: x = df.drop("heart disease", axis=1)
y = df["heart disease"]

[6]: numerical_features = x.select_dtypes(
    include=["int64", "float64"]
).columns
categorical_features = x.select_dtypes(
    include=["object"]
).columns

print("numerical features:")
print(list(numerical_features))

print("categorical features:")
print(list(categorical_features))

numerical_features:
['age', 'cholesterol', 'blood pressure', 'heart rate', 'exercise hours', 'stress level', 'blood sugar']
categorical_features:
['gender', 'smoking', 'alcohol intake', 'family history', 'diabetes', 'obesity', 'exercise induced angina', 'chest pain type']

[7]: x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data:", x_train.shape)
print("Testing Data:", x_test.shape)

Training Data: (286, 15)
Testing Data: (120, 15)

[8]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

[9]: x_train_processed = preprocessor.fit_transform(x_train)
x_test_processed = preprocessor.transform(x_test)

[10]: k = 5
knn = KNeighborsClassifier(
    n_neighbors=k
)

knn.fit(
    x_train_processed,
    y_train
)

[11]: y_pred = knn.predict(
    x_test_processed
)

print("Predicted values:")
print(y_pred)

C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the
number of physical cores for the following reason:
[Error 2] the system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
warnings.warn(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical
_cores
    cpu_info = subprocess.run(
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1446, in run
    with Popen(*popenargs, **kwargs) as process:
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1408, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
.....

Predicted values:
[1 0 0 1 0 1 0 0 0 0 1 1 0 0 1]

[12]: print(
    classification_report(
        y_test,
        y_pred
    )
)
```

precision recall f1-score support

	precision	recall	f1-score	support
0	0.00	0.00	0.00	222
1	0.00	0.00	0.00	222

localhost:8889/notebooks/7.%20K%20Nearest%20Neighbour/KNN%20Classifier.ipynb

Jupyter KNN Classifier Last Checkpoint: 3 days ago

```
[10]: x_train_processed = preprocessor.fit_transform(x_train)
x_test_processed = preprocessor.transform(x_test)

[11]: k = 5
knn = KNeighborsClassifier(
    n_neighbors=k
)

knn.fit(
    x_train_processed,
    y_train
)

[12]: y_pred = knn.predict(
    x_test_processed
)

print("Predicted values:")
print(y_pred)

C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the
number of physical cores for the following reason:
[Error 2] the system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
warnings.warn(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical
_cores
    cpu_info = subprocess.run(
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1446, in run
    with Popen(*popenargs, **kwargs) as process:
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1408, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
.....

Predicted values:
[1 0 0 1 0 1 0 0 0 0 1 1 0 0 1]

[13]: print(
    classification_report(
        y_test,
        y_pred
    )
)
```

precision recall f1-score support

	precision	recall	f1-score	support
0	0.00	0.00	0.00	222
1	0.00	0.00	0.00	222

```
jupyter KNN Classifier Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
precision_values = []
recall_values = []
f1_values = []

for k in k_values:
    knn = neighbors.KNeighborsClassifier(
        n_neighbors=k
    )

    knn.fit(
        X_train_processed,
        y_train
    )

    y_pred = knn.predict(
        X_test_processed
    )

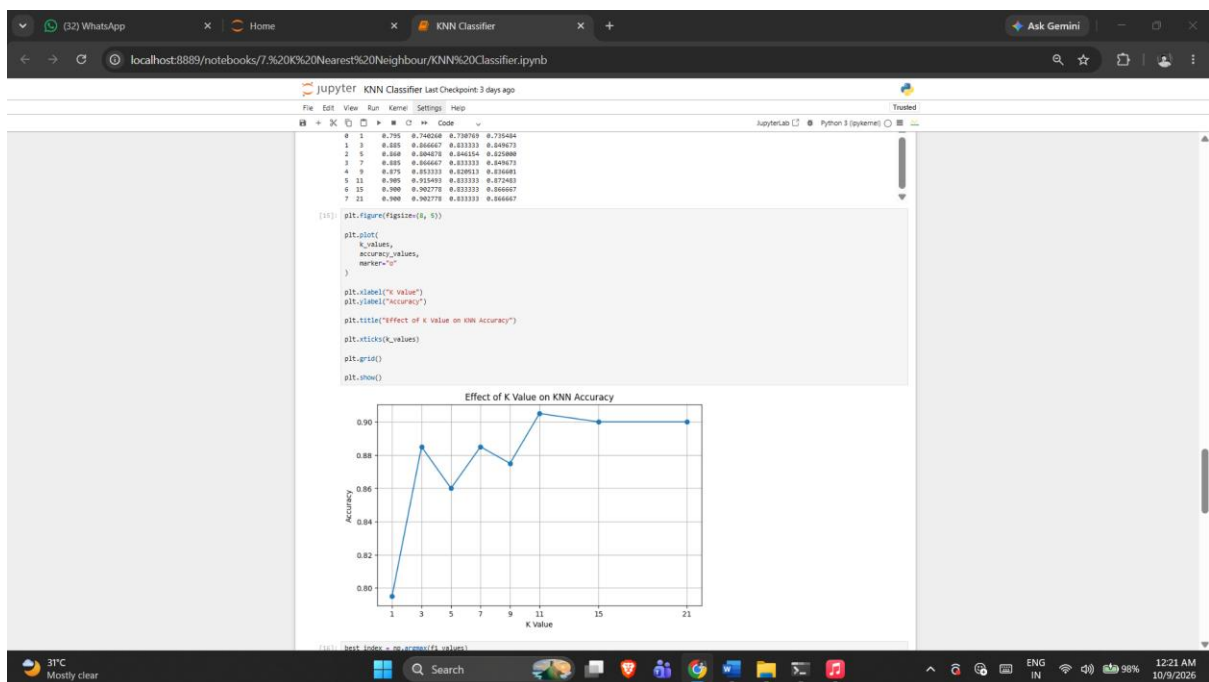
    accuracy_values.append(
        accuracy_score(
            y_test,
            y_pred
        )
    )

    precision_values.append(
        precision_score(
            y_test,
            y_pred
        )
    )

    recall_values.append(
        recall_score(
            y_test,
            y_pred
        )
    )

    f1_values.append(
        f1_score(
            y_test,
            y_pred
        )
    )

[14]: results = pd.DataFrame(
    {
        "k": k_values,
        "accuracy": accuracy_values,
        "precision": precision_values,
        "recall": recall_values,
        "f1_score": f1_values
    }
)
```



5. RESULT

Thus, the K Nearest Neighbour practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -8

Decision Tree Classification

1. AIM

To build a Decision Tree classifier for heart disease prediction and evaluate and visualize the trained model.

2. ALGORITHM

1. Load the dataset and separate features and target.
2. Identify numerical and categorical features.
3. Build the preprocessing pipeline.
4. Split the data into training and testing sets.
5. Train the Decision Tree classifier.
6. Predict and calculate the notebook's evaluation metrics.
7. Visualize the trained tree.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

df = pd.read_csv("heart_disease_dataset.csv")

print("First 5 rows:")
print(df.head())
```

```

print("\nDataset Shape:")
print(df.shape)

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(y.name)

categorical_columns = X.select_dtypes(
    include=["object"]
).columns

numerical_columns = X.select_dtypes(
    exclude=["object"]
).columns

print("\nCategorical Columns:")
print(list(categorical_columns))

print("\Numerical Columns:")
print(list(numerical_columns))

preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,

```

```

        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

model.fit(X_train, y_train)

print("\nDecision Tree Model Trained Successfully!")

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\n=====")
print("          DECISION TREE EVALUATION")
print("=====")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score   :", round(f1, 4))

# =====
# 12. Classification Report
# =====

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# 13. Confusion Matrix
# =====

```

```

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Heart Disease", "Heart Disease"]
)

disp.plot()

plt.title("Confusion Matrix - Decision Tree")
plt.show()

# Get the trained classifier
tree_model = model.named_steps["classifier"]

# Get feature names after One-Hot Encoding
feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()

plt.figure(figsize=(25, 12))

plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["No Heart Disease", "Heart Disease"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Heart Disease Classification")

plt.show()

```

4. OUTPUT

```
localhost:8889/notebooks/8.%20Decision%20Tree%20Classification/Decision%20Tree.ipynb

jupyter Decision Tree Last checkpoint 3 days ago

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

[3]: data = 'Dataset/heart.csv'
df = pd.read_csv(data)

print("First 5 rows:")
print(df.head())
print(df.shape)
print(df.columns)

First 5 rows:
Age  gender  cholesterol  blood_pressure  heart_rate  smoking  \
0    75  female         220             110           60  current  \
1    48  male          284             160           62  current  \
2    53  male          234             95           67  never   \
3    69  female        192             80           72  current  \
4    62  female        172             93           93  never   \

alcohol_intake  exercise_hours  family_history  diabetes  obesity  \
0             heavy             5             NO       NO       YES  \
1             NaN              5             NO       NO       NO  \
2             heavy             3             YES       NO       YES  \
3             NaN              4             NO       YES       NO  \
4             NaN              6             NO       YES       NO  \

stress_level  blood_sugar  exercise_induced  angina  chest_pain_type  \
0             0           110              YES  atypical  angina  \
1             1             70              YES  typical  angina  \
2             5           136              YES  atypical  angina  \
3             7           187              YES non-atypical  pain  \

31°C Mostly clear
```

```
localhost:8889/notebooks/8.%20Decision%20Tree%20Classification/Decision%20Tree.ipynb

jupyter Decision Tree Last checkpoint 3 days ago

[4]: y = df["heart_disease"]

print("features:")
print(x.columns)
print("target:")
print(y.name)

features:
Index(['age', 'gender', 'cholesterol', 'blood_pressure', 'heart_rate',
       'smoking', 'alcohol_intake', 'exercise_hours', 'family_history',
       'diabetes', 'obesity', 'stress_level', 'blood_sugar',
       'exercise_induced_angina', 'chest_pain_type'],
      dtype='object')

target:
heart_disease

[5]: categorical_columns = x.select_dtypes(
    include="object"
).columns

numerical_columns = x.select_dtypes(
    exclude="object"
).columns

print("categorical columns:")
print(list(categorical_columns))

print("numerical columns:")
print(list(numerical_columns))

categorical columns:
['gender', 'smoking', 'alcohol_intake', 'family_history', 'diabetes', 'obesity', 'exercise_induced_angina', 'chest_pain_type']

numerical columns:
['age', 'cholesterol', 'blood_pressure', 'heart_rate', 'exercise_hours', 'stress_level', 'blood_sugar']

[6]: preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        ),
        (
            "num",
            "passthrough"
        )
    ]
)

[7]: decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)
```

Decision Tree Last checkpoint 3 days ago

```
[7]: decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

[8]: model = Pipeline(
    steps=(
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    )
)

[9]: model = Pipeline(
    steps=(
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    )
)

[10]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("training data:", X_train.shape)
print("testing data:", X_test.shape)

training data: (800, 15)
testing data: (200, 15)

[11]: model.fit(X_train, y_train)

print("\nDecision tree model trained successfully!")

Decision tree model trained successfully!

[12]: y_pred = model.predict(X_test)

[13]: accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
```

Decision Tree Last checkpoint 3 days ago

```
X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("training data:", X_train.shape)
print("testing data:", X_test.shape)

training data: (800, 15)
testing data: (200, 15)

[11]: model.fit(X_train, y_train)

print("\nDecision tree model trained successfully!")

Decision tree model trained successfully!

[12]: y_pred = model.predict(X_test)

[13]: accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\n=====")
print("      DECISION TREE EVALUATION")
print("=====")

print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 score  :", round(f1, 4))

=====
DECISION TREE EVALUATION
Accuracy : 0.8
```

Decision Tree Last checkpoint 3 days ago

```
print("Accuracy: ", round(accuracy, 4))
print("Precision: ", round(precision, 4))
print("Recall: ", round(recall, 4))
print("F1 score: ", round(f1, 4))

=====
DECISION TREE EVALUATION
=====
Accuracy : 1.0
Precision : 1.0
Recall : 1.0
F1 Score : 1.0

[14]: # =====
# 22. Classification Report
# =====

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# 23. Confusion Matrix
# =====

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

Classification Report:
              precision    recall  f1-score   support

     0       1.00        1.00        1.00        122
     1       1.00        1.00        1.00         70

 accuracy: 1.00
 avg prec: 1.00
 weighted avg: 1.00

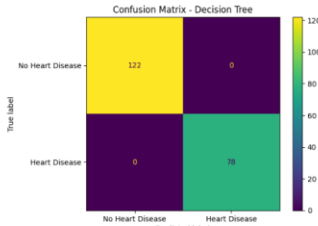
Confusion matrix:
[[122  0]
 [ 0  70]]

[15]: disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Heart Disease", "Heart Disease"])
disp.plot()

plt.title("Confusion matrix - Decision Tree")
```

31°C Mostly clear 12:23 AM 10/9/2026

Decision Tree Last checkpoint 3 days ago



```
[14]: # Get the trained classifier
tree_model = model.named_steps["classifier"]

# Get feature names after One-hot Encoding
feature_names = model.named_steps[
    "imputer"].get_feature_names_out()

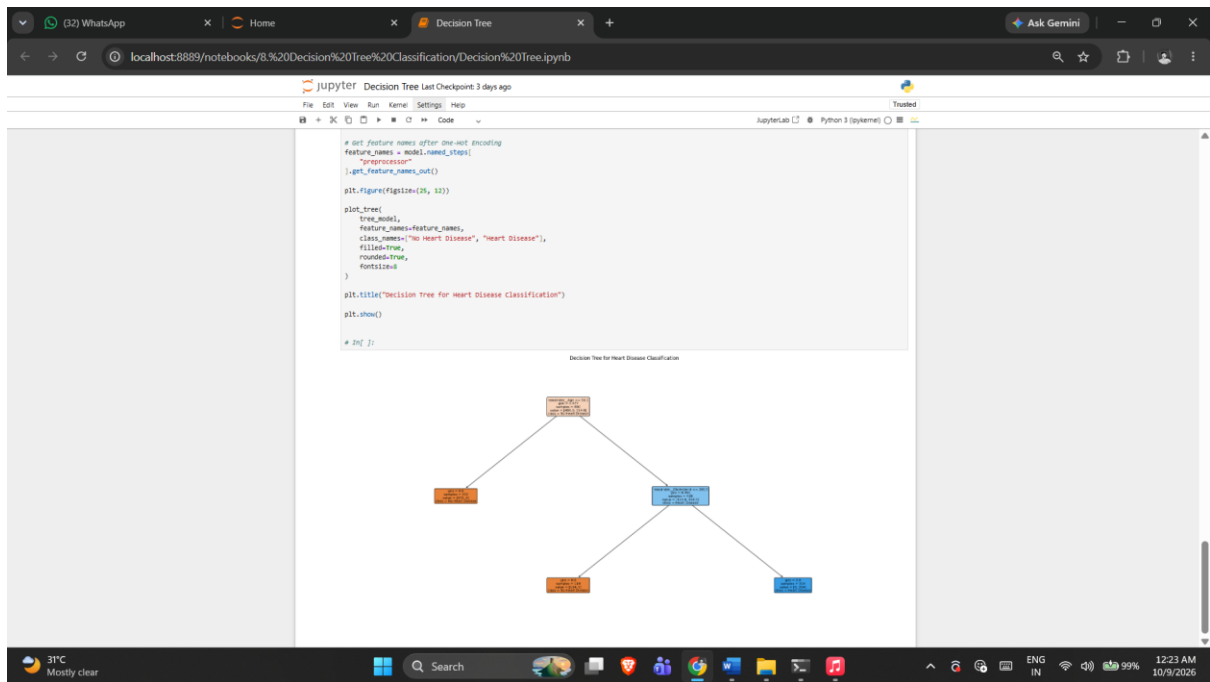
plt.figure(figsize=(25, 12))

plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["No Heart Disease", "Heart Disease"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Heart Disease Classification")
plt.show()

# 21[ ]:
```

31°C Mostly clear 12:23 AM 10/9/2026



5. RESULT

Thus, the Decision Tree Classifier practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -9

Random Forest Classification

1. AIM

To build and evaluate a Random Forest classifier for heart disease prediction, including ROC-AUC analysis.

2. ALGORITHM

1. Load and inspect the dataset.
2. Check structure, data types, missing values and target distribution.
3. Separate features and target.
4. Prepare categorical preprocessing.
5. Create the Random Forest classifier with notebook parameters.
6. Split, train and predict.
7. Calculate classification metrics.
8. Generate ROC curve and ROC-AUC.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

# Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning libraries
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
```

```

)

# Load the dataset

df = pd.read_csv("heart_disease_dataset.csv")

# Display first 5 rows
df.head()

# Display number of rows and columns

print("Dataset Shape:", df.shape)

# Display column names

print(df.columns)

# Display data types

df.info()

# Check missing values

print(df.isnull().sum())

# Check target class distribution

print(df["Heart Disease"].value_counts())

# X contains input features
# y contains the target variable

X = df.drop("Heart Disease", axis=1)

y = df["Heart Disease"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)

# Identify numerical columns

numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

# Identify categorical columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

print("Numerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")

```

```

print(list(categorical_features))

# Preprocessing for numerical and categorical features

preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numerical_features),

        ("cat", OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)

# Create Random Forest classifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)

# Combine preprocessing and Random Forest model

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", rf_model)
    ]
)

# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# Train the model

model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully!")

# Predict on training data

y_train_pred = model.predict(X_train)

```

```

# Predict on testing data

y_test_pred = model.predict(X_test)

print("Predictions Generated Successfully!")

# Calculate evaluation metrics

train_accuracy = accuracy_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)

precision = precision_score(y_test, y_test_pred)

recall = recall_score(y_test, y_test_pred)

f1 = f1_score(y_test, y_test_pred)

print("Training Accuracy :", round(train_accuracy, 4))
print("Testing Accuracy  :", round(test_accuracy, 4))
print("Precision         :", round(precision, 4))
print("Recall            :", round(recall, 4))
print("F1 Score          :", round(f1, 4))

# Predict probabilities

y_test_proba = model.predict_proba(X_test)[:, 1]

# Calculate ROC curve

fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

# Calculate AUC

auc_score = roc_auc_score(y_test, y_test_proba)

print("ROC-AUC Score:", round(auc_score, 4))

# Plot ROC Curve

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label=f"Random Forest (AUC = {auc_score:.2f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

```

```
plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - Random Forest")

plt.legend()

plt.show()
```

4. OUTPUT

The screenshot shows a JupyterLab window with a notebook titled "Random Forest". The code in the first two cells is as follows:

```
[1]: #!/usr/bin/env python
# coding: utf-8

[2]: # Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning libraries
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
)

[3]: # Load the dataset
df = pd.read_csv("Dataset/heart.csv")

# Display first 5 rows
df.head()
```

The output of cell [3] shows the first 5 rows of the heart.csv dataset:

	Age	Gender	Cholesterol	Blood Pressure	Heart Rate	Smoking	Alcohol Intake	Exercise Hours	Family History	Diabetes	Obesity	Stress Level	Blood Sugar	Exercise Induced Angina	Chest Pain Type	Heart Disease
0	75	Female	228	119	66	Current	Heavy	1	No	No	Yes	8	119	Yes	Atypical Angina	1
1	48	Male	204	165	82	Current	N/A	5	No	No	No	9	70	Yes	Typical Angina	0
2	53	Male	234	91	87	Never	Heavy	3	Yes	No	Yes	5	196	Yes	Atypical Angina	1
3	69	Female	192	90	72	Current	N/A	4	No	Yes	No	7	107	Yes	Non-anginal Pain	0
4	62	Female	172	163	93	Never	N/A	6	No	Yes	No	2	183	Yes	Asymptomatic	0

The screenshot shows the same JupyterLab window with the notebook titled "Random Forest". The code in the next two cells is as follows:

```
[3]: print("Dataset shape:", df.shape)
Dataset shape: (1000, 16)

# Display column names
print(df.columns)

Index(['Age', 'Gender', 'Cholesterol', 'Blood Pressure', 'Heart Rate',
       'Smoking', 'Alcohol Intake', 'Exercise Hours', 'Family History',
       'Diabetes', 'Obesity', 'Stress Level', 'Blood Sugar',
       'Exercise Induced Angina', 'Chest Pain Type', 'Heart Disease'],
      dtype='object')

[4]: # Display data types
df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 1000 entries, 0 to 999
Data columns (total 16 columns):
 #   column                Non-Null Count  Dtype
---  -
 0   Age                   1000 non-null    int64
 1   Gender                1000 non-null    object
 2   Cholesterol            1000 non-null    int64
 3   Blood Pressure         1000 non-null    int64
 4   Heart Rate            1000 non-null    int64
 5   Smoking               1000 non-null    object
 6   Alcohol Intake         666 non-null     object
 7   Exercise Hours         1000 non-null    int64
 8   Family History         1000 non-null    object
 9   Diabetes               1000 non-null    object
10   Obesity               1000 non-null    object
11   Stress Level           1000 non-null    int64
12   Blood Sugar            1000 non-null    int64
13   Exercise Induced Angina 1000 non-null    object
14   Chest Pain Type        1000 non-null    object
15   Heart Disease          1000 non-null    int64
dtypes: int64(1), object(15)
memory usage: 125.1+ KB

[5]: # Check missing values
print(df.isnull().sum())

Age                0
Gender             0
Cholesterol        0
Blood Pressure     0
Heart Rate         0
Smoking           360
Alcohol Intake     334
Exercise Hours     0
Family History     0
Diabetes           0
Obesity            0
Stress Level       0
Blood Sugar        0
```

WhatsApp Home Random Forest Ask Gemini

localhost:8889/notebooks/9.%20Random%20Forest%20Classifier/Random%20Forest.ipynb

jupyter Random Forest Last Checkpoint 3 days ago

```
File Edit View Run Kernel Settings Help Trusted
Python 3 (ipykernel)
```

```
[10]: # x contains input features
# y contains the target variable
x = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

print("Features Shape:", x.shape)
print("Target Shape:", y.shape)
Features Shape: (1000, 18)
Target Shape: (1000,)
```

```
[11]: # Identify numerical columns
numerical_features = x.select_dtypes(
    include=["int64", "float64"]
).columns

# Identify categorical columns
categorical_features = x.select_dtypes(
    include=["object"]
).columns

print("Numerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")
print(list(categorical_features))

Numerical features:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']

Categorical features:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
```

```
[12]: # Preprocessing for numerical and categorical features

preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)
```

```
[13]: # Create Random Forest classifier
rf_model = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    max_depth=None,
)
```

31°C Mostly clear Search 12:29 AM 10/9/2026

WhatsApp Home Random Forest Ask Gemini

localhost:8889/notebooks/9.%20Random%20Forest%20Classifier/Random%20Forest.ipynb

jupyter Random Forest Last Checkpoint 3 days ago

```
File Edit View Run Kernel Settings Help Trusted
Python 3 (ipykernel)
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)
```

```
[12]: # Create Random Forest classifier
rf_model = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)
```

```
[13]: # Combine preprocessing and Random Forest model

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", rf_model)
    ]
)
```

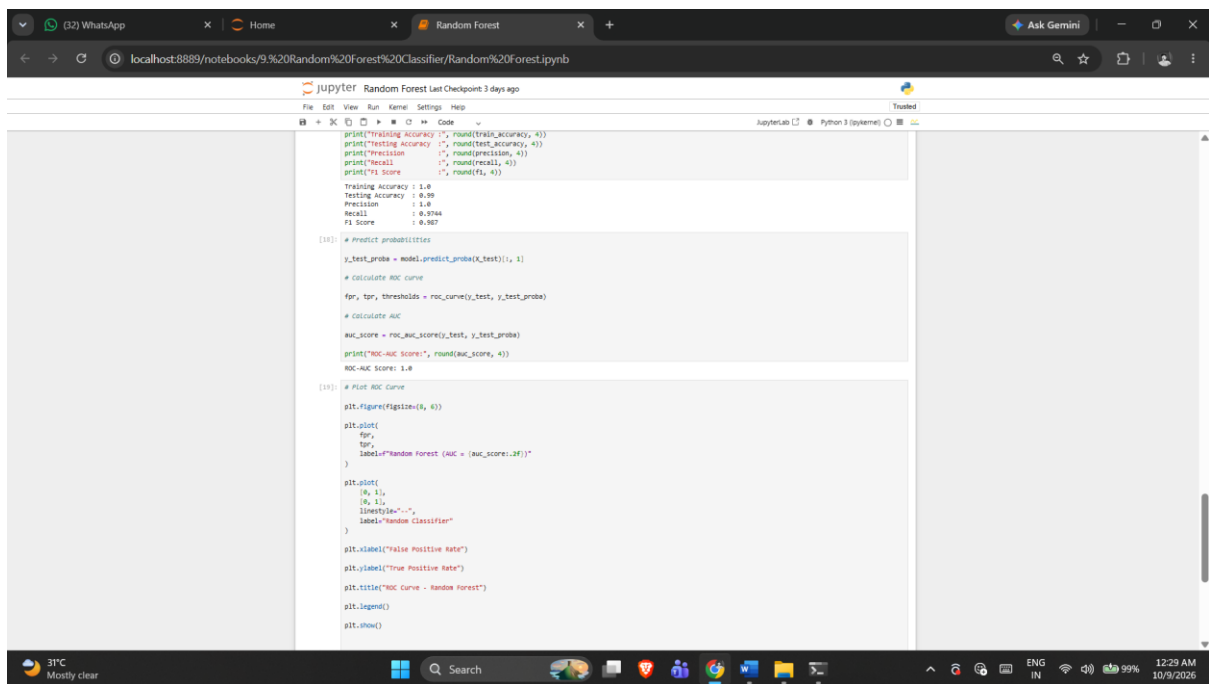
```
[14]: # Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

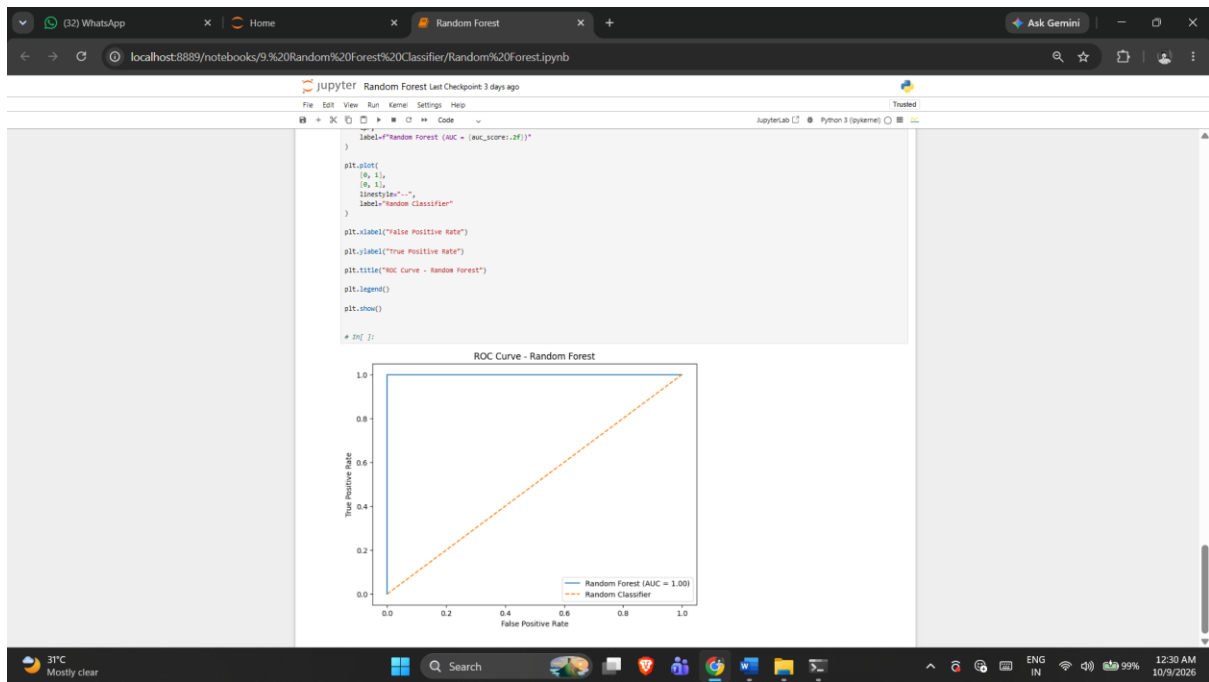
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
Training Data: (800, 18)
Testing Data: (200, 18)
```

```
[15]: # Train the model
model.fit(X_train, y_train)

print("Random Forest Model trained successfully!")
Random Forest Model trained successfully!
```

31°C Mostly clear Search 12:29 AM 10/9/2026





5. RESULT

Thus, the Random Forest Classifier practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -10

Support Vector Machine

1. AIM

To perform SVM classification using Linear, Polynomial, RBF and Sigmoid kernels and compare their performance.

2. ALGORITHM

1. Load the dataset and separate features and target.
2. Identify numerical and categorical features.
3. Create scaling and encoding preprocessing.
4. Split the data with stratification.
5. Train SVM models using the four notebook kernels.
6. Calculate accuracy, precision, recall and F1 score.
7. Compare kernel performance in a table and plot.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
```

```

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

print("\nNumerical Features:")
print(list(numeric_features))

print("\nCategorical Features:")
print(list(categorical_features))

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

kernels = ["linear", "poly", "rbf", "sigmoid"]

results = []

for kernel in kernels:

    model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("svm", SVC(kernel=kernel, random_state=42))
        ]
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results.append([
        kernel,
        accuracy,
        precision,
        recall,
        f1
    ])

print("\n=====")
print("SVM Kernel:", kernel)
print("=====")

```

```

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

results_df = pd.DataFrame(
    results,
    columns=[
        "Kernel",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print("\n\nKernel Comparison:")
print(results_df)

plt.figure(figsize=(8, 5))

plt.bar(
    results_df["Kernel"],
    results_df["Accuracy"]
)

plt.xlabel("SVM Kernel")
plt.ylabel("Accuracy")
plt.title("SVM Performance with Different Kernels")
plt.ylim(0, 1)

plt.show()

# -----
# Result
# -----

print("\nResult:")
print("SVM classification was successfully performed")
print("using Linear, Polynomial, RBF and Sigmoid kernels.")

```

4. OUTPUT

```
ChatGPT | Home | SVM | Ask Gemini | localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb

jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: # Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

[3]: df = pd.read_csv("dataset/heart.csv")
print("Dataset Shape:", df.shape)
print("Dataset 5 records:")
print(df.head())
Dataset Shape: (1000, 16)

First 5 records:
Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 Female 228 119 66 Current
1 48 Male 284 163 62 Current
2 53 Male 284 91 67 Never
3 49 Female 192 90 72 Current
4 42 Female 372 163 93 Never

Alcohol Intake Exercise Hours Family History Diabetes Obesity \
0 heavy 1 NO NO Yes
1 NA 5 NO NO NO
2 heavy 3 YES NO Yes
3 NA 4 NO Yes NO
4 NA 4 NO Yes NO

Stress Level Blood Sugar Exercise Induced Angine Chest Pain Type \
0 0 119 YES Atypical Angina
1 9 70 YES Typical Angina
2 5 196 YES Atypical Angina
3 7 187 YES Non-anginal Pain
4 2 183 YES Asymptomatic

Heart Disease
0 1
1 0
2 1
3 0
4 0
```

```
ChatGPT | Home | SVM | Ask Gemini | localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb

jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

[4]: X = df.drop("heart disease", axis=1)
y = df["heart disease"]

[5]: numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns
print("Numerical Features:")
print(list(numeric_features))
print("Categorical Features:")
print(list(categorical_features))

Numerical Features:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
Categorical Features:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angine', 'Chest Pain Type']

[6]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

[7]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

[8]: kernels = ["linear", "poly", "rbf", "sigmoid"]
results = []

[9]: for kernel in kernels:
    model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("svm", SVC(kernel=kernel, random_state=42))
        ]
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    results.append([
        kernel,
        accuracy,
        precision,
        recall,
        f1
    ])
```

```
ChatGPT | Home | SVM | (2) Terrible time to meet n... | Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb

Jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipykernel)

stop()

("preprocessor", preprocessor),
("svm", SVC(kernel=kernel, random_state=42))
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

results.append([
    kernel,
    accuracy,
    precision,
    recall,
    f1
])

print("=====")
print("SVM kernel:", kernel)
print("=====")

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1 score :", f1)

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

=====
SVM kernel: linear
=====
Accuracy : 0.875
Precision: 0.87229436237103
Recall : 0.79427374671746
F1 score : 0.83224763866712

Classification Report:
              precision    recall  f1-score   support

0               0.88         0.93         0.90         122
1               0.87         0.79         0.83          78

accuracy          0.88         200
macro avg         0.87         0.86         0.87         200
weighted avg      0.87         0.86         0.87         200

=====
new results: null
```

```
ChatGPT | Home | SVM | (2) Terrible time to meet n... | Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb

Jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipykernel)

print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

=====
SVM kernel: linear
=====
Accuracy : 0.875
Precision: 0.87229436237103
Recall : 0.79427374671746
F1 score : 0.83224763866712

Classification Report:
              precision    recall  f1-score   support

0               0.88         0.93         0.90         122
1               0.87         0.79         0.83          78

accuracy          0.88         200
macro avg         0.87         0.86         0.87         200
weighted avg      0.87         0.86         0.87         200

=====
SVM kernel: poly
=====
Accuracy : 0.920
Precision: 0.93386849338684
Recall : 0.871794273746718
F1 score : 0.9026233386292

Classification Report:
              precision    recall  f1-score   support

0               0.92         0.96         0.94         122
1               0.93         0.87         0.90          78

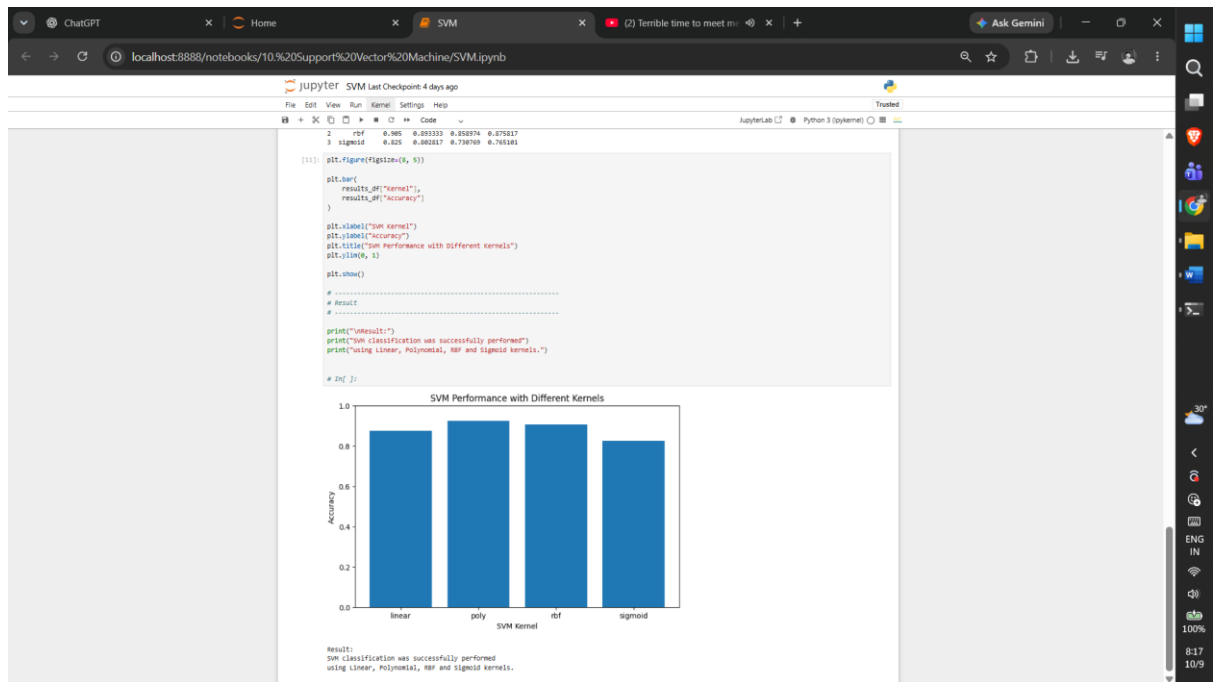
accuracy          0.93         200
macro avg         0.93         0.92         0.92         200
weighted avg      0.93         0.93         0.92         200

=====
SVM kernel: rbf
=====
Accuracy : 0.905
Precision: 0.893333333333333
Recall : 0.858746386554859
F1 score : 0.87583893444823

Classification Report:
              precision    recall  f1-score   support

0               0.91         0.93         0.92         122
1               0.89         0.86         0.88          78

accuracy          0.90         200
macro avg         0.90         0.90         0.90         200
weighted avg      0.90         0.91         0.90         200
```



5. RESULT

Thus, the Support Vector Machine practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -11

K Means Clustering

1. AIM

To perform K-Means clustering and use the Elbow Method to determine a suitable number of clusters and visualize the resulting groups.

2. ALGORITHM

1. Load and prepare the dataset.
2. Preprocess the features as implemented in the notebook.
3. Run K-Means for the specified range of K values.
4. Calculate the clustering inertia values.
5. Plot the Elbow curve.
6. Fit and visualize the selected clustering solution.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import KMeans

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)
```

```

)

X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()

print("\nProcessed Data Shape:")
print(X_processed.shape)

# -----
# Step 7: Apply Elbow Method
# -----

inertia = []

K = range(1, 11)

for k in K:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_processed)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(
    K,
    inertia,
    marker="o"
)

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")

plt.xticks(K)

plt.grid(True)

plt.show()

# Change this value after observing the elbow plot.
optimal_k = 3

print("\nSelected Optimal K:", optimal_k)

kmeans = KMeans(
    n_clusters=optimal_k,

```

```

        random_state=42,
        n_init=10
    )

clusters = kmeans.fit_predict(X_processed)

df["Cluster"] = clusters

print("\nClustered Dataset:")
print(df.head(10))

print("\nNumber of records in each cluster:")
print(df["Cluster"].value_counts().sort_index())

```

4. OUTPUT

```

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import KMeans

[3]: df = pd.read_csv("dataset/heart.csv")

print("Dataset Shape:", df.shape)
print("First 5 records:")
print(df.head())

Dataset Shape: (1000, 16)

First 5 records:
   age  gender  cholesterol  blood pressure  heart rate  smoking  \
0    75   Female      228          129         66    Current
1    40   Male      284          125         62    Current
2    53   Male      234          95          67     Never
3    69   Female     192          96          75    Current
4    42   Female     172          163          85     Never

   alcohol  intake  exercise hours  family history  diabetes  obesity  \
0    Heavy      1          1        No      No      Yes
1    None      5          0        No      No      No
2    Heavy      3          0      Yes      No      Yes
3    None      4          0      No      Yes      No
4    None      6          0      No      Yes      No

   stress level  blood sugar  exercise induced  angina  chest pain type  \
0          6          119          Yes      Physical angina
1          9          76          Yes      Typical angina
2          5          196          Yes      Atypical angina
3          7          187          Yes      Non-anginal pain
4          2          183          Yes      Asymptomatic

   heart disease
0          1
1          0
2          1
3          0
4          0

[4]: X = df.drop("heart disease", axis=1)

[5]: numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

[6]: preprocessor = ColumnTransformer(

```

```
jupyter K-MEANS CLUSTERING WITH ELBOW METHOD Last Checkpoint: 4 days ago

# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Load the dataset
heart_disease = pd.read_csv('heart_disease.csv')

# Drop the 'heart_disease' column
X = heart_disease.drop('heart_disease', axis=1)

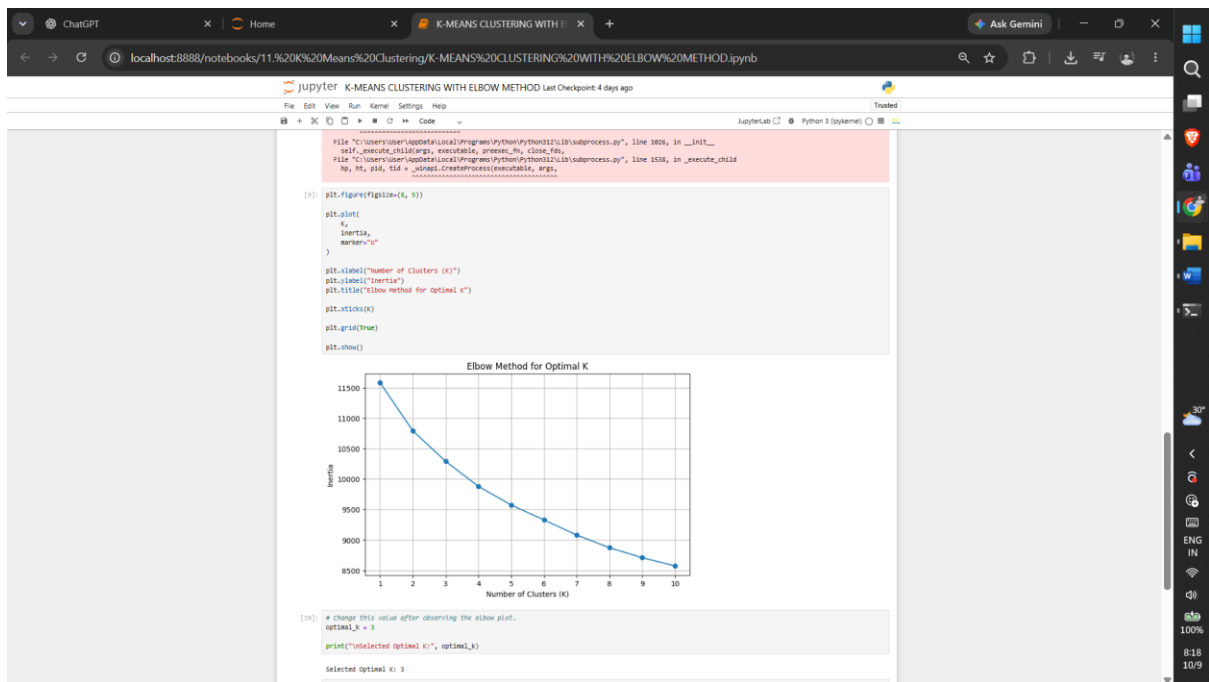
# Select numeric and categorical features
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object']).columns

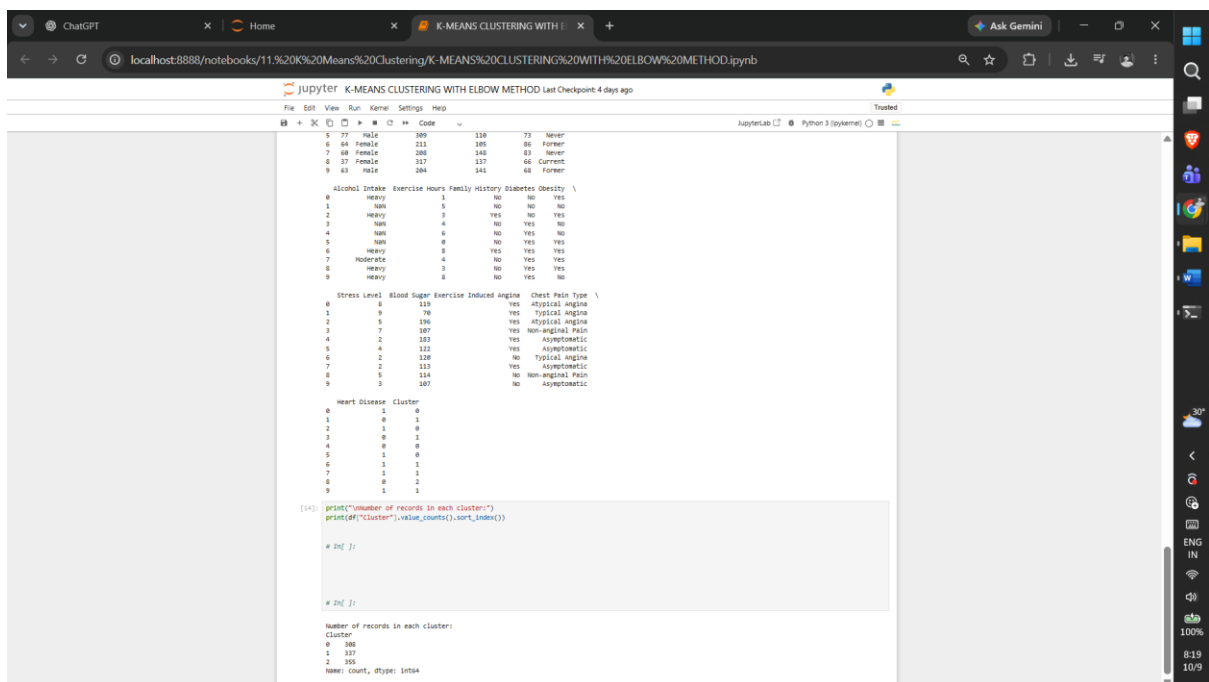
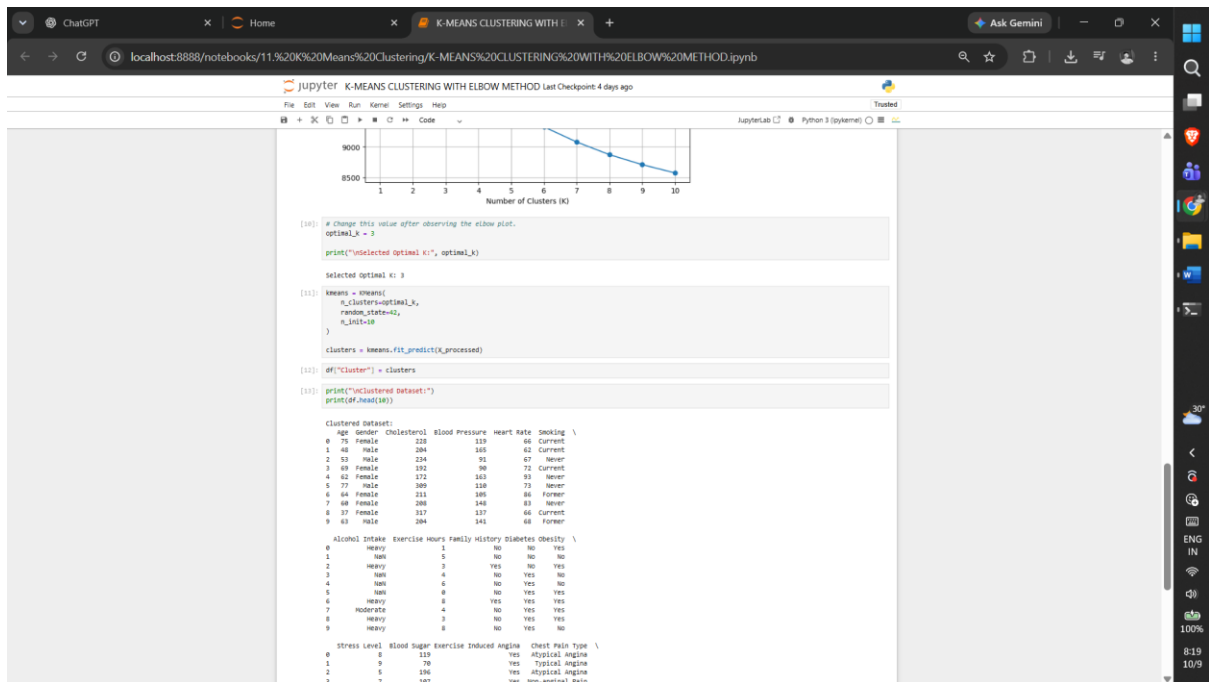
# Create a ColumnTransformer to preprocess the data
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ]
)
X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a numpy array
if hasattr(X_processed, 'toarray'):
    X_processed = X_processed.toarray()
print('X_processed data shape:')
print(X_processed.shape)

# Step 2: Apply Elbow Method
inertia = []
for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        init='k-means++'
    )
    kmeans.fit(X_processed)
    inertia.append(kmeans.inertia_)

# Plot the inertia values for different numbers of clusters
plt.figure(figsize=(10, 5))
plt.plot(
    range(1, 11),
    inertia,
    marker='o'
)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal K')
plt.grid(True)
plt.show()
```





5. RESULT

Thus, the K Means Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -12

Hierarchical Clustering

1. AIM

To perform hierarchical clustering on the preprocessed heart disease dataset using Ward linkage and visualize the hierarchy with a dendrogram.

2. ALGORITHM

1. Load the dataset and remove the target.
2. Identify numerical and categorical features.
3. Standardize numerical data and one-hot encode categorical data.
4. Create the processed feature matrix.
5. Perform Ward hierarchical linkage.
6. Plot the dendrogram.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from scipy.cluster.hierarchy import (
    dendrogram,
    linkage
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
```

```

transformers=[
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
]

X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()

print("\nProcessed Data Shape:")
print(X_processed.shape)

linked = linkage(
    X_processed,
    method="ward"
)

plt.figure(figsize=(14, 7))

dendrogram(
    linked,
    truncate_mode="lastp",
    p=30,
    leaf_rotation=90,
    leaf_font_size=10,
    show_contracted=True
)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Cluster / Data Points")
plt.ylabel("Distance")

plt.show()

```

4. OUTPUT

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from scipy.cluster.hierarchy import (
    dendrogram,
    linkage
)

df = pd.read_csv("dataset/heart.csv")

print("Dataset Shape", df.shape)
print("First 5 records")
print(df.head())

Dataset Shape: (1000, 16)

First 5 records:
Age  gender  Cholesterol  Blood Pressure  Heart Rate  Smoking \
0  75  female      228          119         66  Current
1  60  male       284          145         62  Current
2  53  male       234          95         67  Never
3  69  female     192          96         72  Current
4  62  female     172          163         95  Never

Alcohol Intake  Exercise Hours  Family History  Diabetes  Obesity \
0  heavy         1             1             NO         NO  YES
1  NaN           5             NO             NO         NO  NO
2  heavy         3             YES            NO         YES
3  NaN           4             NO            YES         NO
4  NaN           6             NO            YES         NO

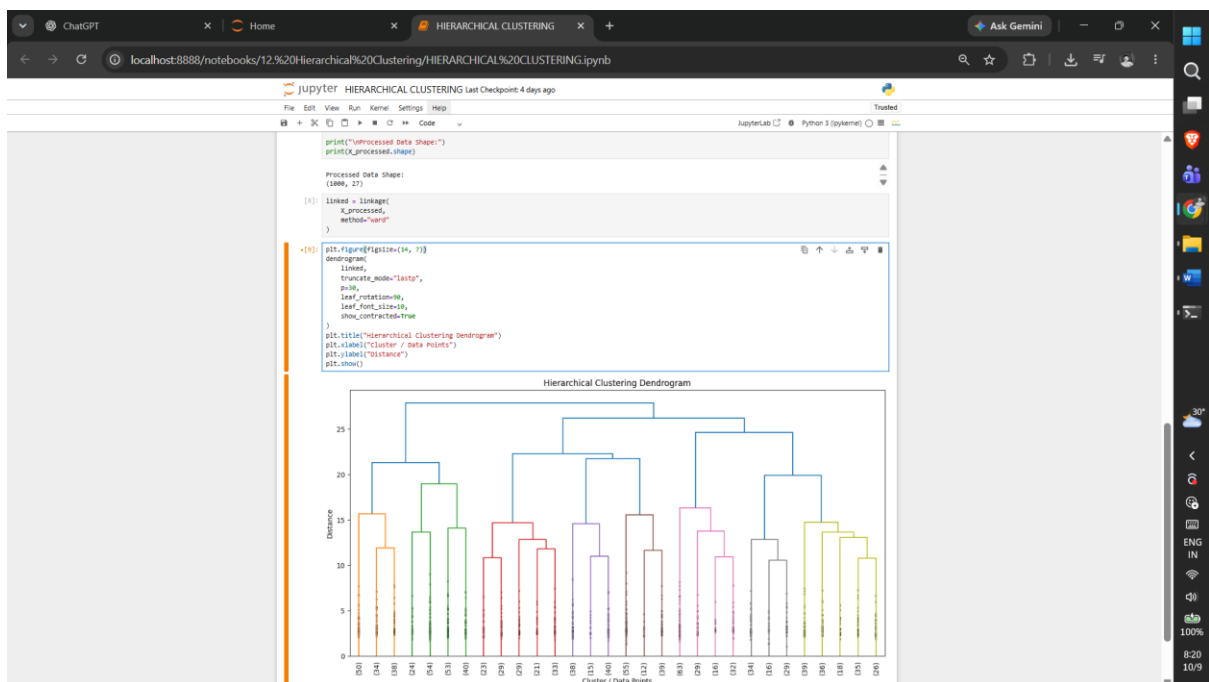
Stress Level  Blood Sugar  Exercise Induced  Angina  Chest Pain Type \
0  6             119        YES              Physical Angina
1  9             70         YES              Typical Angina
2  5             196        YES              Physical Angina
3  7             187        YES              Non-anginal Pain
4  2             183        YES              Asymptomatic

Heart Disease
0  1
1  0
2  1
3  0
4  0

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
```



5. RESULT

Thus, the Hierarchical Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -13

DBSCAN Clustering

1. AIM

To perform DBSCAN clustering on the heart disease dataset, identify clusters and noise points, and visualize the clustering result using PCA.

2. ALGORITHM

1. Load the dataset and separate the target.
2. Identify numerical and categorical features.
3. Scale numerical features and one-hot encode categorical features.
4. Apply DBSCAN using the notebook parameters.
5. Assign labels, identify noise points and count clusters.
6. Apply PCA for two-dimensional visualization.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
```

```

    ]
)

X_processed = preprocessor.fit_transform(X)

X_processed = X_processed

dbscan = DBSCAN(
    eps=1.5,
    min_samples=5
)

labels = dbscan.fit_predict(X_processed)

df["Cluster"] = labels

print("\nCluster Labels:")
print(df["Cluster"].value_counts().sort_index())

noise_points = df[df["Cluster"] == -1]

print("\nNumber of Noise Points:")
print(len(noise_points))

print("\nNoise Points:")
print(noise_points.head(10))

number_of_clusters = len(
    set(labels)
) - (1 if -1 in labels else 0)

print("\nNumber of Clusters:")
print(number_of_clusters)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_processed)

plt.figure(figsize=(9, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=labels,
    cmap="viridis",
    s=40
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("DBSCAN Clustering")

plt.colorbar(label="Cluster")

plt.show()

```

4. OUTPUT

```
jupyter DBSCAN CLUSTERING AND NOISE ANALYSIS Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help Trusted
Python 3 (system)

[1]: #!/usr/bin/env python
# coding: utf-8

[2]: import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

[3]: df = pd.read_csv("dataset/heart.csv")
print("dataset shape:", df.shape)
print("first 5 records:")
print(df.head())
dataset shape: (1000, 16)

first 5 records:
Age  gender  Cholesterol  blood pressure  heart rate  smoking \
0  75  female    258        130          80    Current
1  48  male      204        105          65    Current
2  53  male      234        95           65    never
3  49  female    192        90          72    Current
4  42  female    172        160          95    never

alcohol intake  exercise hours  family history  diabetes  obesity \
0  heavy         1           1          no         yes
1  na            5           0          no         no
2  heavy         3           1          no         yes
3  na            4           0          no         yes
4  na            6           0          no         no

stress level  blood sugar  exercise induced  angina  chest pain type \
0  0           119        yes                atypical  angina
1  9           70        yes                atypical  angina
2  5           196        yes                atypical  angina
3  7           167        yes                non-anginal pain
4  2           183        yes                asymptomatic

heart disease
0  1
1  0
2  1
3  0
4  0

[4]: X = df.drop("heart disease", axis=1)

[5]: numeric_features = X.select_dtypes(
    include=["int64", "float64"])
categorical_features = X.select_dtypes(
    include="object")
numeric_features = numeric_features.dropna()
categorical_features = categorical_features.dropna()
```

```
jupyter DBSCAN CLUSTERING AND NOISE ANALYSIS Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help Trusted
Python 3 (system)

[6]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ])
X_processed = preprocessor.fit_transform(X)

[7]: X_processed = X_processed

[8]: dbSCAN = DBSCAN(
    eps=0.5,
    min_samples=5)

labels = dbSCAN.fit_predict(X_processed)

C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\site-packages\joblib\external\loky\backend\context.py:136: UserWarning: Could not find the
number of physical cores for the following reason:
[liberror 2] the system cannot find the file specified
Determining the number of logical cores instead. You can silence this warning by setting LOCKING_CPU_COUNT to the number of cores you want to use.
warnings.warn(
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\site-packages\joblib\external\loky\backend\context.py", line 267, in _count_physical
cores
cpu_info = subprocess.run(
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\subprocess.py", line 940, in run
with Popen(*popenargs, **kwargs) as process:
.....
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\subprocess.py", line 1406, in _init_
self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\subprocess.py", line 1508, in _execute_child
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
.....

[9]: df["cluster"] = labels

[10]: print("Number of Noise Points:")
print(len(noise_points))
print(noise_points)

Cluster Labels:
Cluster
-1  9000
Name: count, dtype: int64

[11]: noise_points = df[df["cluster"] == -1]
print("Number of Noise Points:")
print(len(noise_points))
print(noise_points)

Number of Noise Points:
1000

Noise Points:
.. .. .. .. ..
```

```
DBSCAN CLUSTERING AND NOISE ANALYSIS Last checkpoint: 4 days ago

print("Noise Points:")
print(noise_points.head(10))

Number of Noise Points:
1000

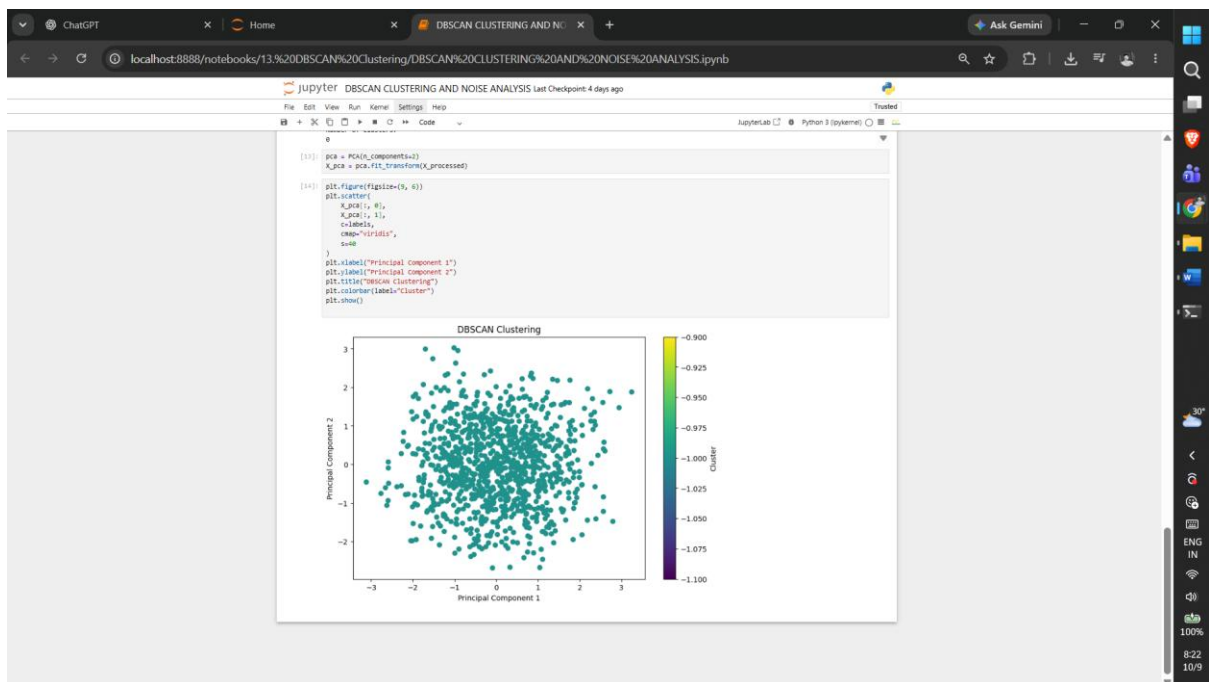
Noise Points:
  Age  Gender  Cholesterol  Blood Pressure  Heart Rate  Smoking
0  75  Female      220          119          66  Current
1  40  Male      204          105          62  Current
2  53  Male      234          95           63  Never
3  69  Female     192          90          72  Current
4  42  Female     172          83          59  Never
5  77  Male      309          138          73  Never
6  64  Female     211          105          86  Former
7  68  Female     200          140          83  Never
8  37  Female     117          117          60  Current
9  63  Male      204          141          68  Former

  Alcohol Intake  Exercise Hours  Family History  Diabetes  Obesity
0  Heavy         1              No             No        Yes
1  None          5              No             No        No
2  Heavy         3              Yes            No        Yes
3  None          4              No            Yes        No
4  None          6              No            Yes        No
5  None          0              No            Yes        Yes
6  Heavy         8              Yes            Yes        Yes
7  Moderate      4              No            Yes        Yes
8  Heavy         3              No            Yes        Yes
9  Heavy         8              No            Yes        No

  Stress Level  Blood Sugar  Exercise Induced Angina  Chest Pain Type
0  8           110         Yes                    atypical Angina
1  9           70          Yes                    typical Angina
2  5           140         Yes                    atypical Angina
3  7           107         Yes                    non-anginal Pain
4  4           103         Yes                    asymptomatic
5  4           122         Yes                    asymptomatic
6  2           120         No                     typical Angina
7  2           113         Yes                    asymptomatic
8  5           114         No                    non-anginal Pain
9  5           107         No                    asymptomatic

  Heart Disease  Cluster
0  1             -1
1  0             -1
2  1             -1
3  0             -1
4  0             -1
5  1             -1
6  1             -1
7  1             -1
8  0             -1
9  1             -1

[11]: number_of_clusters = len(
      set(labels)
      ) - (1 if -1 in labels else 0)
print("Number of Clusters:")
```



5. RESULT

Thus, the DBSCAN Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -14

Principal Component Analysis

1. AIM

To apply Principal Component Analysis to reduce the preprocessed heart disease feature space to two components and measure explained variance.

2. ALGORITHM

1. Load the dataset and separate the target.
2. Identify numerical and categorical features.
3. Scale numerical features and one-hot encode categorical features.
4. Convert the processed data when required.
5. Fit PCA with two components.
6. Transform the data and calculate explained variance.
7. Plot the two principal components.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
```

```

        transformers=[
            ("num", StandardScaler(), numeric_features),
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
        ]
    )

X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()

print("\nProcessed Data Shape:")
print(X_processed.shape)

pca = PCA(
    n_components=2
)

X_pca = pca.fit_transform(X_processed)

pca_df = pd.DataFrame(
    X_pca,
    columns=[
        "Principal Component 1",
        "Principal Component 2"
    ]
)

print("\nPCA Transformed Data:")
print(pca_df.head())

print("\nExplained Variance Ratio:")

print(pca.explained_variance_ratio_)

total_variance = pca.explained_variance_ratio_.sum()

print("\nTotal Variance Explained:")
print(total_variance)

print(
    "\nPercentage of Variance Explained:",
    total_variance * 100,
    "%"
)

plt.figure(figsize=(9, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    s=40
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

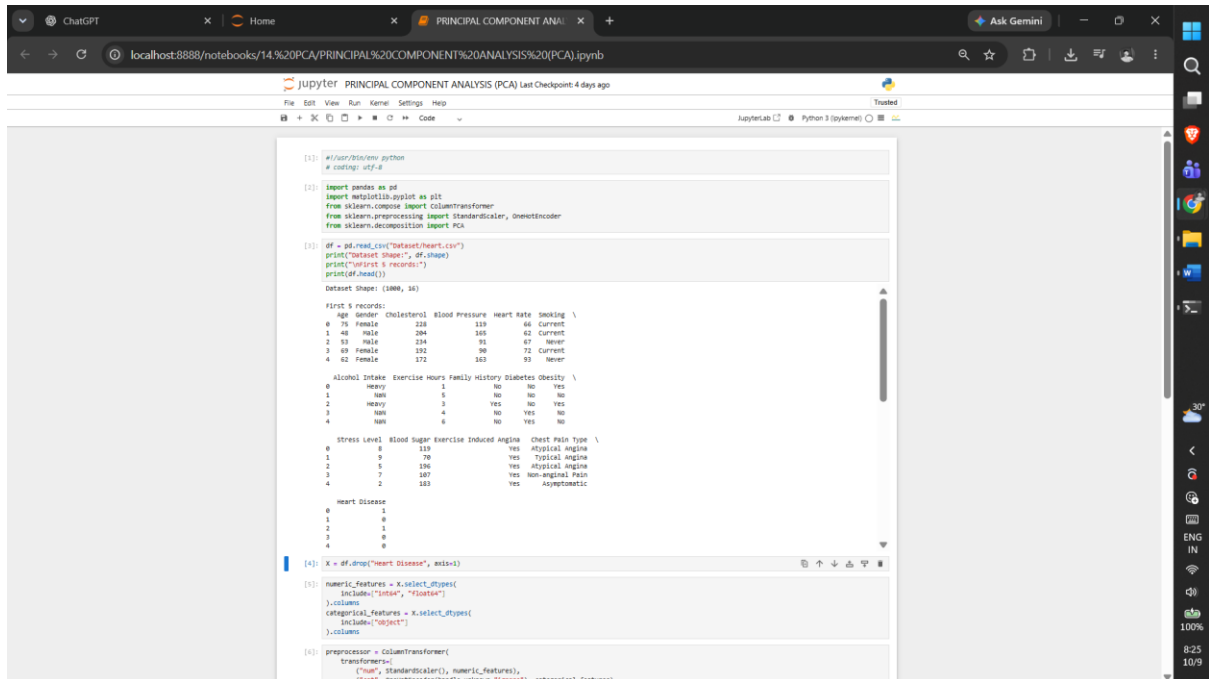
```

```
plt.title("PCA Visualization")
```

```
plt.grid(True)
```

```
plt.show()
```

4. OUTPUT



The screenshot shows a JupyterLab notebook titled "PRINCIPAL COMPONENT ANALYSIS (PCA)". The code in the first cell includes imports for pandas, sklearn.preprocessing, and sklearn.decomposition. It reads a CSV file named "dataset/heart.csv", prints its shape (1000, 16), and displays the first 5 records. The data includes columns for Age, Gender, Cholesterol, Blood Pressure, Heart Rate, Smoking, Alcohol Intake, Exercise Hours, Family History, Diabetes, Obesity, Stress Level, Blood Sugar, Exercise Induced Angina, Chest Pain Type, and Heart Disease. The second cell shows the selection of features: numeric features (Age, Cholesterol, Blood Pressure, Heart Rate, Blood Sugar, Exercise Induced Angina, Chest Pain Type) and categorical features (Gender, Smoking, Diabetes, Obesity, Exercise Induced Angina, Chest Pain Type). A ColumnTransformer is used to preprocess the data, handling numeric and categorical features separately.

```
[1]: # Import libraries
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA

[2]: df = pd.read_csv("dataset/heart.csv")
print("Dataset Shape:", df.shape)
print("First 5 records:")
print(df.head())

Dataset Shape: (1000, 16)

First 5 records:
Age  Gender  Cholesterol  Blood Pressure  Heart Rate  Smoking  \
0  75  Female  220        119           66  Current  \
1  48  Male  204        105           62  Current  \
2  57  Male  234        95           67  Never  \
3  69  Female  192           90           72  Current  \
4  42  Female  172        163           95  Never

Alcohol Intake  Exercise Hours  Family History  Diabetes  Obesity  \
0  Heavy          1           1  No  No  Yes  \
1  No            5           0  No  No  No  \
2  Heavy          3           3  Yes  No  Yes  \
3  No            4           0  No  Yes  No  \
4  No            8           0  No  Yes  No

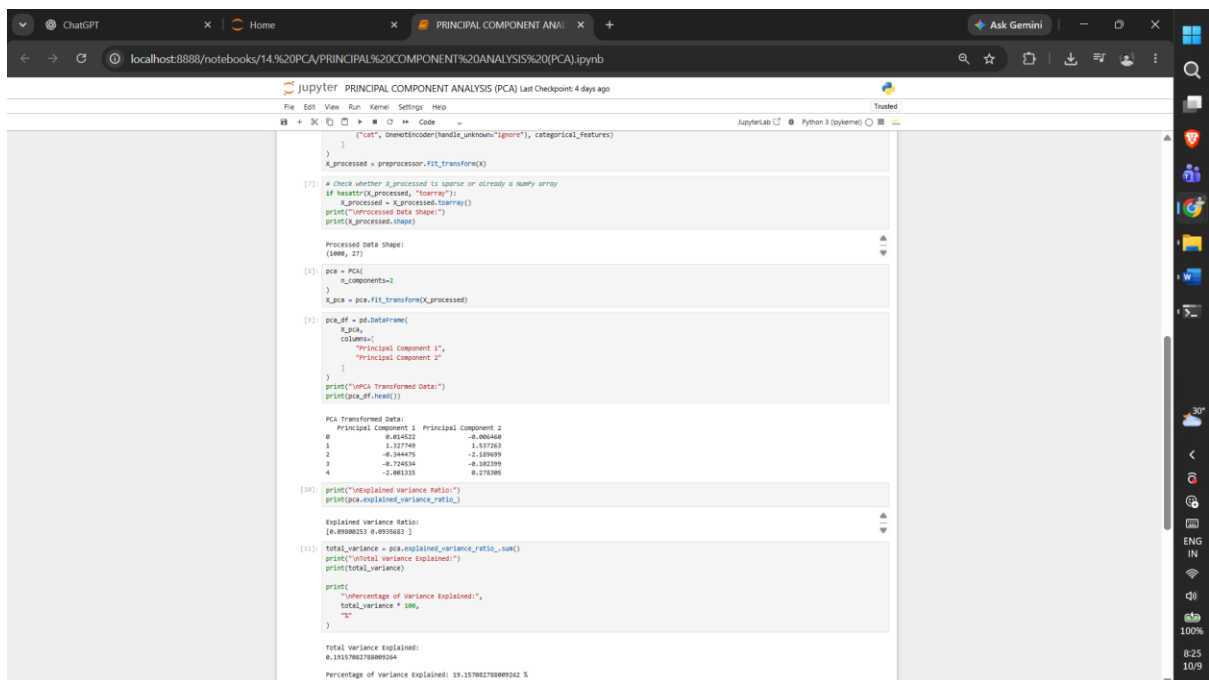
Stress Level  Blood Sugar  Exercise Induced Angina  Chest Pain Type  \
0  0           119           Yes  Atypical Angina  \
1  9           70           Yes  Typical Angina  \
2  5           106           Yes  Atypical Angina  \
3  7           187           Yes  Non-anginal Pain  \
4  2           183           Yes  Asymptomatic

Heart Disease
0  1
1  0
2  1
3  0
4  0

[3]: X = df.drop("Heart Disease", axis=1)

[4]: numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
    include=["object"]
).columns

[5]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
```



The screenshot shows the continuation of the PCA analysis. The third cell completes the ColumnTransformer definition. The fourth cell fits the transformer on the data X. The fifth cell checks if the transformed data is sparse or already a NumPy array and prints its shape. The sixth cell creates a PCA object with 2 components and fits it to the transformed data. The seventh cell prints the PCA transformed data. The eighth cell prints the explained variance ratio. The ninth cell calculates the total variance explained and prints it. The tenth cell prints the percentage of variance explained.

```
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
    X_processed = preprocessor.fit_transform(X)

[6]: # Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()
print("Unprocessed data shape:")
print(X_processed.shape)

Processed Data Shape:
(1000, 20)

[7]: pca = PCA(
    n_components=2
)
X_pca = pca.fit_transform(X_processed)

[8]: pca_df = pd.DataFrame(
    X_pca,
    columns=[
        "Principal Component 1",
        "Principal Component 2"
    ]
)
print("PCA Transformed Data:")
print(pca_df.head())

PCA Transformed Data:
Principal Component 1  Principal Component 2
0  0.814522  -0.400468
1  1.127740  -1.157703
2  -0.344470  -2.189039
3  -0.734634  -0.340199
4  -2.401135  0.270305

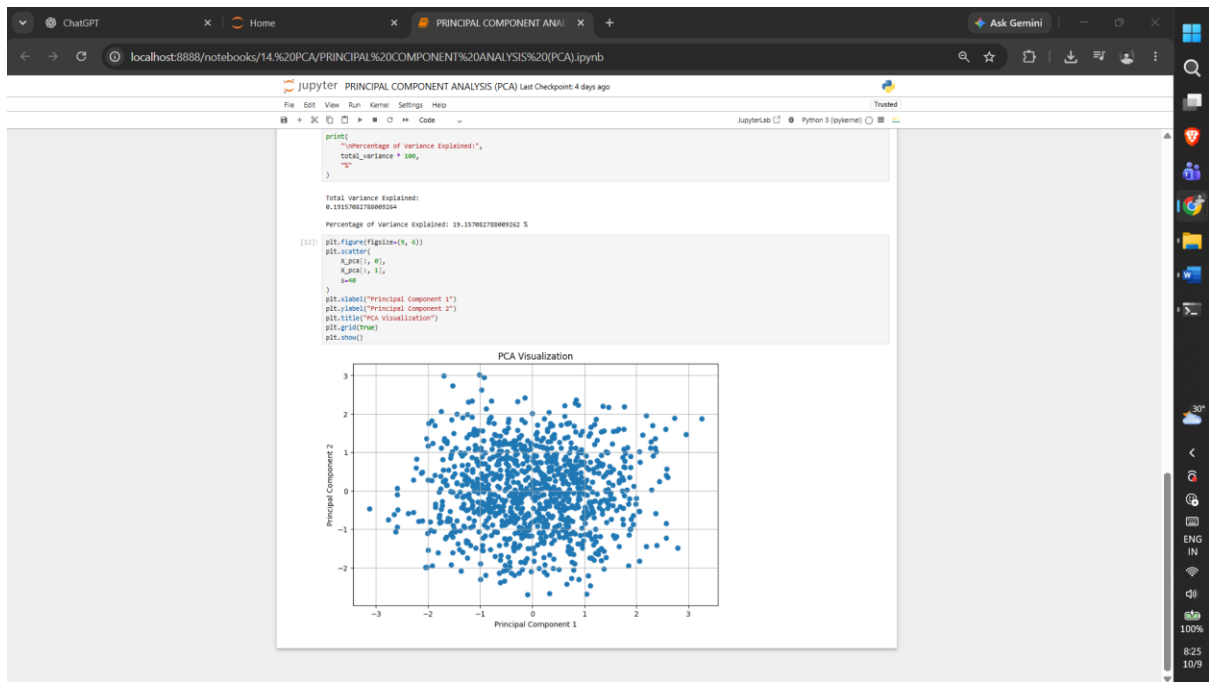
[9]: print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

Explained Variance Ratio:
[0.49800253 0.49356623]

[10]: total_variance = pca.explained_variance_ratio_.sum()
print("Total Variance Explained:")
print(total_variance)

print(
    "Percentage of Variance Explained:",
    total_variance * 100,
    "%"
)

Total Variance Explained:
0.19157002700000004
Percentage of Variance Explained: 19.157002700000002 %
```



5. RESULT

Thus, the PCA practical was implemented successfully using the program from the corresponding Jupyter Notebook.